# What Keeps a Generative Recommender Relevant? An Empirical Study of Reinforcement Fine-Tuning Against a Learned Reward Model

Daniil Antonov

ITMO University, Saint Petersburg, Russia

Draft, version 4  15 October 2026

**Abstract**

A recommender that generates a list item by item can be fine-tuned with reinforcement learning (RL) towards feedback that next-item training does not see. We study the least protected form of this idea, in which a learned model of user feedback is the only source of reward: a GPT-2 recommender is fine-tuned with a policy gradient to raise the consumed share of its items as a second GPT-2 predicts it, held near the pretrained recommender by a KL anchor, on a video log (KION) and a music log (Yambda-50M), with three seeds for every central comparison. Instruments that the optimised model cannot influence judge the result: a reward model fitted on other users, the logged response to the recommended items that users went on to consume, the same lists scored for another user, and relevance split by outcome, by user and by position in the list. We answer five questions. *Behaviour:* the policy learns which items everybody finishes, not what each user wants; on KION a predicted gain of 24% is almost entirely impersonal, and the logged share per recommended item rises by 1%. *Relevance and variety:* on KION NDCG@10 falls by 11.5%, most for users with short histories, and impressions concentrate further on the most shown titles; on Yambda both are kept. *Long-term goals:* rewards for return, likes and adoption raise their predictions by a few percent at a moderate distance from the pretrained model, and the one large gain, the return after the list, comes from drifting to short titles. *Several steps ahead:* a generated list gains after a hit and loses after a miss, and the reward of later steps depends on an assumption about responses nobody observed, which a reward model trained with hidden responses makes unbiased. *Trade-off:* RL settings choose a point on one frontier, and only the reward function moves it; re-ranking by the same reward model reaches more reward at equal relevance. All results are offline.

**Keywords:** generative recommendation, sequential recommendation, reinforcement learning, reward model, reward hacking, offline evaluation.

## 1 Introduction

Sequential recommenders built as autoregressive transformers over item or semantic identifiers [1], [2], [3], [4], [5] are trained to continue logged sequences. The log records what users did, not what served them: in the two logs studied here, 36% of music listens stop before half of the track and 52% of video titles are abandoned before half of their length. A recommender that generates a list item by item is also a policy that takes a sequence of decisions, so reinforcement learning (RL) is the natural tool for moving it from "what comes next" towards "what will be received well" [6], [7], [8], [9].

Where the reward comes from decides how safe that step is. Two recent studies that report gains without a loss of relevance both keep the reward tied to the log. Petrov and Macdonald [8] fine-tune GPTRec with PPO on its own generated lists, with a reward that contains NDCG against the user's true next item. Matveev et al. [9] train a retriever with off-policy REINFORCE on logged sessions, with logged feedback as the reward, and use a learned feedback model only to evaluate. The third option is to let a learned model of user feedback *be* the reward. It is attractive because such a model scores any item for any user, so the policy can be trained on its own generations and for signals that the log holds only for the items users chose. It is also the option with the fewest protections: a policy optimised against a learned model drifts to where the model is wrong [10], [11], [12], [13], and in recommendation the drift takes the form of popularity and filter-bubble effects [14], [15], [16].

This paper studies the third option in its barest form. A GPT-2 [17] next-item recommender is fine-tuned with a policy gradient to raise the share of an item that a second GPT-2, the *reward model*, predicts the user will consume. Nothing in this reward asks for relevance; a KL penalty towards the pretrained recommender, the *anchor*, is the only term that does. The study answers five questions, which organise its results:

**Q1** How does behaviour change, and why? (Section 5.3)
**Q2** Is relevance kept? Does everyone get the same items? (Section 5.4)
**Q3** Long-term goals: return and continued interaction. (Section 5.5)
**Q4** Generating several steps ahead. (Section 5.6)
**Q5** Trade-off between relevance and reward. (Section 5.7)

Table 1 answers each question in one sentence; the sections named above give the evidence.

Table 1: The five questions and their answers. Numbers are from KION unless Yambda is named; "±" is the standard deviation over three seeds.

| Question | Answer in one sentence |
|---|---|
| **Q1.** How does behaviour change, and why? | The policy learns which titles everybody finishes rather than what each user wants: on KION it moves impressions from long series to short films, 78% of its change is one shift per item, and nearly all of the 24% predicted gain would be the same for any other user, because the reward model's predictions differ between items far more than between users' reactions to one item. |
| **Q2.** Is relevance kept? Does everyone get the same items? | On KION relevance is not fully kept (NDCG@10 $-11.5\% \pm 1.3$, most for users with short histories) and impressions concentrate further (the most shown 1% of titles take 76% of them instead of 72%), while on Yambda both are kept; in both, the KL distance from the pretrained recommender, not the anchor weight, decides how much is kept, and without the anchor every user gets the same list. |
| **Q3.** Long-term goals: return and continued interaction. | Return, likes and adoption can be predicted and rewarded, but at a moderate distance their predictions rise by a few percent (adoption: +19% with no loss of relevance and a fifth less coverage), and the one large gain, the return after the list on KION (+18%), comes from drifting to short titles at a 69% loss of NDCG@10; none of it can be confirmed offline. |
| **Q4.** Generating several steps ahead. | A generated list gains after a hit and loses after a miss (on Yambda the next generated item is a hit in 54% of lists after a hit against 25% in a ranked list, and in 4% against 6% after a miss), the predicted reward of later steps depends on an assumption about responses nobody observed, which a reward model trained with hidden responses makes unbiased, and crediting an item with the rest of the list helps no per-item reward. |
| **Q5.** Trade-off between relevance and reward. | RL settings choose a point on the frontier; only the reward function moves the frontier: within 1% of the base recommender's NDCG@10, RL on the share gains 4% of predicted share on KION, the same RL with logged relevance in the reward 17%, and re-ranking by the same reward model 15%. |

Every reward gain in this setting is reported by a model. The pipeline therefore carries instruments that do not depend on the model being optimised, and the study is organised around what they show. Our contributions are the following.

- **A protocol for judging RL against a learned reward offline.** A second reward model fitted on a disjoint half of the users; the logged response to those recommended items that the user went on to consume, which no model stands behind; the same lists scored for another user, which separates what the policy learned about the user from what it learned about items; relevance split by the logged outcome, by the length of the user's history and by position in the list; and the reward model's predictions on real sequences under each assumption a simulation has to make. Operating points are selected on validation requests only (Section 3.5, Section 4).
- **Three seeds for every central comparison.** The main run, the anchor sweep, the KL budget, the estimator and credit comparisons, the alternatives to RL and logged relevance in the reward were each trained with three seeds, and every central conclusion holds in each of them (Section 5.2).
- **What the policy learns, and why.** On KION 78% of the change in the policy's scores is one shift per item, correlated 0.90 with the item's usual consumed share; 0.2% of the predicted gain depends on who receives the list. The anchor weight is not a comparable quantity, the KL distance is, and at an equal distance the estimator and the credit assignment make no difference (Section 5.3, Section 5.4).
- **Long-term rewards and several steps ahead.** Rewards for return, likes and adoption, a reward paid after the whole list, hits by position of generated and ranked lists, and a reward model trained with hidden responses that removes the bias of simulated feedback (Section 5.5, Section 5.6).

- **The frontier.** Every RL setting we varied moves the policy along one frontier of relevance against reward; adding logged relevance to the reward moves the frontier, and re-ranking by the same reward model reaches more of the reward at equal relevance without training (Section 5.7).

All results are offline. Section 6.1 designs the experiment that would settle what they leave open, and Section 7 lists the rest.

# 2 Related Work

**Generative recommenders and RL.** GRU4Rec [18], SASRec [1] and BERT4Rec [19] made next-item prediction the default formulation of sequential recommendation; generative recommenders use the same models autoregressively, over items [2] or semantic identifiers [3], up to industrial scale [4], [5], [20]. We keep the simplest member of the family so that the effects of fine-tuning are not confounded with tokenisation. RL enters recommendation through off-policy policy gradients [6], slate values [7], [21] and RL heads on supervised models [22], [23]. The two studies we build on sit at opposite ends: on-policy PPO over the recommender's own generations with a reward read from the true next item [8], and off-policy REINFORCE over logged sessions [9]. Learned models enter rewards when the wanted signal is sparse [24] and replace the user altogether in simulators [25], [26], [27]; ours is a simulator learned from the log the recommender was trained on.

**Learned rewards and their exploitation.** Model-based offline RL trusts a model only where data supports it [10], [11], [28]. For language models fine-tuned against reward models, a "gold" reward first rises and then falls while the proxy keeps rising [12]; ensembles mitigate this [29], and reward hacking has a formal treatment [13]. Our held-out reward model plays the part of the gold model, and we find that it is not gold enough: two models trained on the same kind of data share what they have learned about items. In recommendation, conservative offline RL amplifies the Matthew effect [14] and feedback loops degenerate [15], [16].

**Constraints and rewards.** TRPO and PPO bound each update relative to the previous iterate [30], [31]; the adaptive-penalty variant of PPO adjusts a KL coefficient towards a target divergence, the rule that our KL budget applies to the pretrained policy. Penalising divergence from a *pretrained* model is KL-control [32], [33], used when language models are fine-tuned from human preferences [34], [35] and as behaviour regularisation in offline RL [36]. Consumption time is confounded by item duration [37], [38]; we observe the confounding from the policy's side and remove it with a within-item quantile. Long-term engagement and retention have been optimised directly [39], [40] and through proxies [41], [42]. The diversity of a list is an objective that no single item can earn; maximal marginal relevance [43] is its greedy optimiser and our reference for a list-dependent reward.

**Generation and evaluation.** Feeding a model its own outputs takes it away from the inputs it was trained on [44], [45], and spending inference compute on several candidates that a learned model judges is the recommender form of test-time scaling [9], [46]. Next-item protocols hide deficiencies of RL recommenders [47], and off-policy estimators such as the doubly robust one [48] need logged propensities, which neither log provides. Results selected and judged by the same feedback model are not independent evidence [9], and baselines need one protocol [49].

# 3 Method

Figure 1 shows the pipeline: a recommender is pretrained on next-item prediction, two reward models are fitted on disjoint halves of the users, and the recommender is fine-tuned on the reward that one of them predicts for its own generated lists.

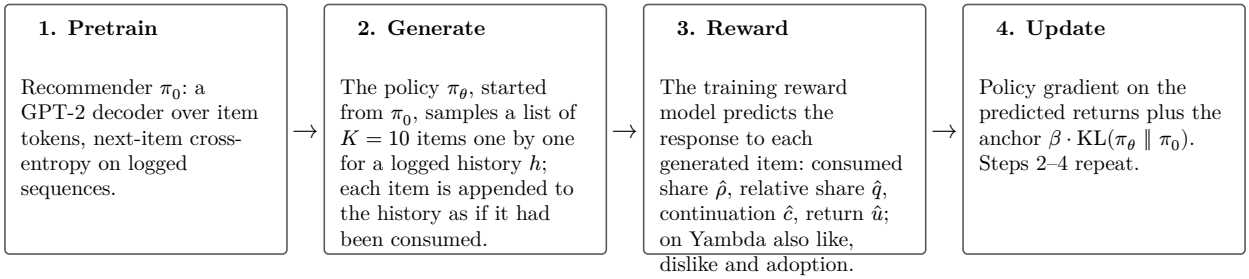



Figure 1: The pipeline. The reward model of step 3 is the only source of reward during fine-tuning; the instruments in the dashed box never influence the policy.

## 3.1 Setting and models

A log is a set of events, each with a user, an item $i_t$, a time and the *consumed share* $\rho_t \in [0, 1]$: the share of the track that was played or of the title that was watched. A recommender is a distribution $\pi_\theta(j \mid h)$ over the next item given the history $h$; $\pi_0$ is the recommender before fine-tuning, and a recommendation is a list $a_{1:K}$ with $K = 10$.

**Base recommender.** A GPT-2 decoder [17] over item tokens with tied input and output embeddings, trained with the full-softmax next-item cross-entropy. A list is obtained by *ranking* (Top-K: the catalogue is scored once) or by *generation* (Next-K: each chosen item is appended to the history as if the user had consumed it, and no item repeats within a list) [2]. Fine-tuning changes the second mode, in which the recommender is a policy over a sequence of decisions.

**Reward model.** A second GPT-2 decoder of the same size. The token at position $t$ carries the item $i_t$ together with the consumed share of the *previous* item in ten buckets, so the state at position $t$ has seen the candidate item and the whole history with its responses, but not the response it predicts. A generated item can thus be scored before any outcome for it exists. Heads read the state, each trained with binary cross-entropy: the consumed share $\hat{\rho}$ (against $\rho$ as a soft label); the *relative share* $\hat{q}$ defined below; the probability $\hat{c}$ that the session continues (the user's next event follows within a gap $g$); the probability $\hat{u}$ that the user returns (a new session starts within a window $w$); and a choice head, which an earlier iteration of the study found useless as a relevance signal and which no reward here uses. A retention outcome counts as known only if its whole waiting time lies inside the observed data.

**Reactions and adoption.** Where the log records them (Yambda), three more heads predict that the user likes the item, dislikes it, and *adopts* it. A like or a dislike is attributed to the user's latest play of the same track within a day before the reaction. A track is adopted when the user's first play of it is followed by another play in a later session (more than the session gap after the first) within 14 days; the label is known only for first plays whose 14 days lie inside the log. These heads enter the loss with the weight of the retention heads, a quarter of the consumed share's.

**Relative share.** The consumed share pays for items that are easy to finish, whoever receives them: a film over a series, a short track over a long one. To separate the user's response from the item's, each event also gets its mid-rank quantile among the training-period events of the same item,

$$q_t = \frac{\#\{\rho' < \rho_t\} + \frac{1}{2}\#\{\rho' = \rho_t\}}{n_{i_t}}, \tag{1}$$

where $\rho'$ runs over the $n_{i_t}$ training-period consumed shares of item $i_t$. Its mean is exactly one half for every item, so no item is better than another on average, and a policy can raise it only by matching items to the users who take to them more than others do.

**Two copies.** Two reward models are trained on disjoint halves of the users. The *training* reward model produces the rewards during fine-tuning. The *held-out* reward model never influences the policy and only scores its lists.

**Hidden responses.** To score the $k$-th item of a generated list, the reward model needs the responses to the items before it, which nobody observed. A reward model trained only on logged sequences has never read an unknown response. A second pair of reward models is therefore trained with one more input value, "unknown": in half of the training windows the responses to the last 1 to 10 events, chosen uniformly, are replaced by it. This is the situation of a generated list, whose history has known responses and whose generated items do not. The validation loss is computed with all responses visible.

## 3.2 Reinforcement fine-tuning

One episode is the generation of one list for a history sampled from the training period. The state is $s_k = (h, a_{1:k-1})$, the action is the next item $a_k$, the reward $r_k$ is predicted by the training reward model (by default $r_k = \hat{\rho}_k$), and the return is $G_k = \sum_{l \geq k} \gamma^{l-k} r_l$. The reward model is the environment. To score the $k$ -th item it is fed, for the items generated before it, its own expectation $\hat{\rho}$ by default; Section 5.6 compares this with a sampled response and with the "unknown" input.

The policy gradient $\mathbb{E}\left[\sum_k A_k \nabla_\theta \log \pi_\theta(a_k \mid s_k)\right]$ is estimated with one of two advantage estimators. The default uses a learned *value model* $V_\varphi$, a copy of the recommender's transformer with a scalar head, and generalised advantage estimates [50]; each batch of lists is used for four gradient steps on the clipped surrogate of PPO [31]. The alternative is a *leave-one-out* baseline: several lists are sampled for each history, and the baseline of a list is the mean return of the others [51], [52]. Advantages are standardised to zero mean and unit variance within a batch in both cases. The clipping of the surrogate touched at most 0.3% of sampled items in any run that did not collapse, so we do not attribute anything to PPO: in effect the default is an actor–critic policy gradient with several steps per batch.

**Anchor and KL budget.** The loss adds a penalty on the divergence from the frozen pretrained recommender, computed exactly over the catalogue at every position:

$$\mathcal{L}(\theta) = \mathcal{L}_{\mathrm{PG}}(\theta) + \beta \cdot \mathbb{E}_k[\mathrm{KL}(\pi_\theta(\cdot \mid s_k) \parallel \pi_0(\cdot \mid s_k))]. \tag{2}$$

Since the default reward has no relevance term, $\beta$ is the only thing that asks the policy to stay relevant. The penalty is weighed against standardised advantages, so the same $\beta$ is a different constraint under another estimator, reward or dataset. A *KL budget* $\delta$ fixes the constraint itself: after every iteration the weight is multiplied by $\exp\left(\eta\left(\overline{\mathrm{KL}}/\delta - 1\right)\right)$, with $\eta = 0.03$ and the mean divergence $\overline{\mathrm{KL}}$ of the iteration, so that the policy settles at the distance $\delta$ in nats per generated item.

**Credit along the list.** Two ablations test whether fine-tuning uses the sequential structure of a list. With $\gamma = 0$ the return is the reward of the item itself. With *one-item episodes* a single item is sampled per episode, so the policy never learns from a context that contains generated items; at evaluation its lists are still generated item by item.

## 3.3 Rewards

All rewards are instances of

$$\begin{aligned} r_k = {} & w_\rho(\hat{\rho}_k - b_k) + w_q \hat{q}_k + w_c \hat{c}_k + w_u \hat{u}_k + w_L\left(\hat{l}_k - \hat{d}_k\right) + w_A \hat{\alpha}_k \\ & + w_R \mathbb{1}[k = K] \hat{u}_K + w_F \frac{\mathbb{1}[a_k \in F]}{\log_2(k+1)} + \frac{w_D}{K-1} \sum_{l<k} d(a_k, a_l). \end{aligned} \tag{3}$$

The default is $w_\rho = 1$ and everything else zero. The *relative-share* reward is $w_q = 1$ alone, and the retention rewards use $w_c$ and $w_u$. The *like* reward pays the predicted probability of a like minus that of a dislike ($w_L = 1$), the *adoption* reward the predicted probability $\hat{\alpha}$ that the item is adopted ($w_A = 1$). The *return after the list* ($w_R = 1$) pays the predicted return after the last item only, so that the reward belongs to the list as a whole and reaches each item through the return $G_k$. The term with $w_F$ is the *logged-relevance* reward of [8]: for a history from the training period the user's real continuation $F$ is known, and the term sums to DCG@K over the list. A baseline $b_k$ makes the reward *personal*: it subtracts what the same reward model predicts for the same list shown to another user, or the item's mean share in the training period. The last term makes the reward depend on the *list as a whole*: $d$ is the cosine distance between the embeddings of two items in $\pi_0$, and over a list the term sums to $w_D K/2$ times its mean pairwise distance.

### 3.4 Alternatives to on-policy RL

Four methods raise the same rewards without generating lists in a training loop. **Liked-only fine-tuning** continues next-item training for one epoch on targets with $\rho \geq 0.5$; it uses logged feedback and no reward model. **Off-policy REINFORCE** [9] raises the log-probability of every logged item in proportion to the discounted logged reward that followed it in the session, with importance weights against $\pi_0$ capped at $\kappa$. **Re-ranking** takes the 100 best candidates of $\pi_0$ and keeps the $K$ with the highest $r(j \mid h) + w \log \pi_0(j \mid h)$, where $r$ is the reward predicted for $j$ as the next item after the real history; nothing is trained, and $w$ plays the part of the anchor. For the list reward it becomes greedy maximal marginal relevance [43]. **Best of $n$ lists** samples $n$ lists from $\pi_0$ and serves the one with the highest predicted return, the test-time scaling of [9].

### 3.5 Instruments

For evaluation, $F$ is the set of the next $M = 20$ items the user really consumed after $h$.

- **Held-out reward model.** A gain that the reward model of the other half of the users does not confirm is exploitation of the errors of the training model.
- **Logged response.** A recommended item that is in $F$, a *hit*, comes with the response the log recorded, and no model stands between that response and the recommender. We report the number of hits per list, their mean logged share, and two quantities per recommended item in which an item that was not hit counts as nothing: the *logged share* $\frac{1}{K}\sum_k \mathbb{1}[a_k \in F]\rho(a_k)$ and the *logged surplus* $\frac{1}{K}\sum_k \mathbb{1}[a_k \in F]\big(q(a_k) - \frac{1}{2}\big)$, which is positive only if users took to the recommended items more than people usually take to those items. Hits are a selected sample of the recommendations: these numbers say how the items the user did reach were received, not how the rest of the list would have been.
- **Another user.** A list generated for a user with history $h$ is also scored for a user with another history $h'$. The gain of a method over $\pi_0$ splits into the gain other users would give the same lists and the rest, whose share of the total is the *personal share of the gain*.
- **Relevance.** NDCG, Recall, MAP and HitRate at $K = 1, 5, 10$ against $F$, and NDCG@10 against its parts: *liked* ($\rho \geq 0.5$) and *skipped* items; items consumed *above* and *below usual* ($\rho$ against the item's mean share in the training period); and, where repetition is allowed, *repeats* from the visible history and *new* items. Users are also grouped into quartiles of the number of events before the request.
- **Concentration.** Catalogue coverage (the share of items in at least one list), the Gini coefficient of the impressions per item, the share of impressions taken by the 1% most shown items, and the expected overlap between the lists of two users (*pair overlap*).
- **Per-item shift.** For each history the change $\log \pi_\theta(j \mid h) - \log \pi_0(j \mid h)$ of every item splits into the item's mean change and the rest; the share of its variance that the item means explain, weighted by $\pi_0$, is 1 for a policy that applies one preference per item to every user.
- **Position.** The rate at which the item at each position of a list is a hit, overall, after a miss and after a hit at the position before.
- **Simulation on real sequences.** For requests with at least five next events, the list is the user's real next five items in order, and the reward model predicts the share of each from the history and the items before it, with the earlier responses as logged, or as a simulation would have to assume them.

## 4 Experimental Setup

**Data.** Yambda-50M [53] is a listening log of Yandex Music: each event has the track, a timestamp, the played share, the track length and a flag telling whether the user chose the track or a recommender served it; separate tables hold likes and dislikes. KION [54] is a log of an online cinema with one row per user and title: the date of the last viewing and the watched share of the title; an item table gives each title's name, type (film or series), year and genres. Both logs are split along one time axis: the last 7 days are the test period, the 7 days before them the validation period, the rest the training period; the catalogue and all item statistics come from the training period only (Table 2). The consumed share is bimodal in both logs, so the reward model is in effect a skip classifier. On KION the share depends strongly on the length of the title: films are watched to 0.5 on average, and the longest tenth of the catalogue, which is series, to 0.22, because the share is taken of the whole series. A reward built on the share prefers films before any model is involved. On Yambda the session continues if the user has another event within 30 minutes and the user

Table 2: The two processed logs.

| | KION | Yambda-50M |
|---|---|---|
| Users | 256,407 | 8,711 |
| Items in the catalogue | 7,992 | 50,000 |
| Events, millions (training / validation / test) | 3.51 / 0.19 / 0.18 | 35.0 / 1.13 / 1.08 |
| Events per user, median | 10 | 2,638 |
| Mean consumed share | 0.50 | 0.65 |
| Events consumed at least half | 48% | 64% |
| Events that continue the session / are followed by a return | 42% / 71% | 95% / 92% |
| Events the user chose (not served by a recommender) | not recorded | 51% |
| Events with a like / a dislike | not recorded | 0.77% / 0.16% |
| First plays adopted (played again in a later session within 14 days) | – | 24% |
| Test requests | 59,997 | 7,593 |

returns by starting a new session within a day; KION only has dates, so a session is a calendar day and a return is a viewing on another day within a week.

**Models and training.** The recommender and the reward models have 4 layers, width 256 and 4 attention heads, with windows of 48 events on KION and 128 on Yambda, trained with AdamW [55] and stopped by the validation loss. Re-recommending items from the history is allowed on Yambda, where repeated listening is normal, and blocked on KION, where every user–title pair occurs once; every method obeys the same constraint. Fine-tuning samples one list for each of 512 histories per iteration ($\gamma = 0.99$, $\lambda = 0.9$, clipping threshold 0.2, learning rates $10^{-5}$ for the policy and $10^{-4}$ for the value model, which is trained alone for the first 50 iterations). The leave-one-out estimator samples 4 lists for each of 128 histories, so both see as many lists per iteration. The *main run* has $\beta = 0.3$ and 2,000 iterations; every other fine-tuning run has 1,000.

**Baselines.** The most popular items, a replay of the user's own history (most played first), a first-order Markov chain, GRU4Rec [18], SASRec with its original loss [1] and BERT4Rec [19]. The neural baselines take the width, depth, dropout and window length of the recommender and are stopped by their own validation loss; none was tuned.

**Evaluation.** Each test user with a history makes one request at the start of the test week: the last 38 events on KION (118 on Yambda) are the history and the next at most 20 events are the ground truth. All quantities are measured on generated lists of ten unless ranked lists are named. The change from the base to the fine-tuned recommender is reported with a 95% interval from a paired bootstrap over users, which measures the noise of the user sample, and with the standard deviation over three seeds, which measures the noise between training runs.

**Selection.** No model, hyperparameter or operating point was chosen on the test period within this protocol. Model sizes and learning rates were fixed before the study. Sweeps are reported in full, and where one point of a sweep has to be named we name the run with the highest held-out predicted share on *validation* requests among the runs that keep the validation NDCG@10 within 5% of the base recommender's. The weight $\beta = 0.3$ of the main run is the exception: it was carried over from earlier iterations of this study, in which test metrics of anchor sweeps had been seen, so it is an illustration and not a selected optimum. By the validation rule the selected weights are $\beta = 1$ on KION and $\beta = 0.05$ on Yambda (Table 25), and the review sessions select the same.

**Sessions and seeds.** Each dataset was run in three kinds of sessions on one 96 GB GPU (Table 23). A *final* session ran the full grid: 49 fine-tuning runs on KION (6.1 hours) and 25 on Yambda, in a first session and a continuation that reused its models (12.3 hours together). A *review* session retrained every model with the same seed and ran the long-term rewards, the reward models with hidden responses, and the analyses by position, user and item (1.7 hours on KION, 7.0 on Yambda); it reproduced the base recommenders of the final sessions exactly (NDCG@10 0.1173 and 0.1172). Two *seed* sessions repeated the core of the study with seeds 1 and 2: the base recommender, both reward models, the main run, the anchor sweep, a KL budget, the credit comparison, the alternatives to RL, the decomposition of the gain and logged relevance in the reward, and on KION also the leave-one-out estimator and the relative-share reward (1.9 hours per seed on KION, 4.5 on Yambda). Seed 0 is the seed of the final and review sessions, and the numbers in the text are seed 0′s unless a standard deviation is given.

Table 3: Baselines on generated or ranked lists of ten for the test requests. The last two rows are the recommender that is fine-tuned, ranked and generated. On KION items of the history are blocked, so every hit is a new item.

| | KION | | | Yambda | | | |
|---|---|---|---|---|---|---|---|
| | **NDCG@10** | **Recall@10** | **Coverage** | **NDCG@10** | **on repeats** | **on new** | **Coverage** |
| Most popular | 0.0903 | 0.1303 | 0.004 | 0.0186 | 0.0193 | 0.0115 | 0.000 |
| Replay of the history | 0.0903 | 0.1303 | 0.004 | 0.1873 | 0.3722 | 0.0000 | 0.519 |
| Markov chain | 0.0959 | 0.1394 | 0.466 | 0.0550 | 0.0934 | 0.0139 | 0.457 |
| GRU4Rec | 0.1215 | 0.1721 | 0.544 | 0.1116 | 0.1706 | 0.0298 | 0.451 |
| SASRec | 0.1082 | 0.1580 | 0.335 | 0.0743 | 0.1070 | 0.0242 | 0.367 |
| BERT4Rec | 0.1290 | 0.1811 | 0.427 | 0.1036 | 0.1717 | 0.0232 | 0.461 |
| GPT-2, ranked | 0.1220 | 0.1726 | 0.574 | 0.1138 | 0.1765 | 0.0315 | 0.466 |
| GPT-2, generated | 0.1173 | 0.1620 | 0.495 | 0.1172 | 0.1865 | 0.0261 | 0.519 |

# 5 Results

In the tables, "Pred. share" is the consumed share of the generated lists as the held-out reward model predicts it, and "Logged" and "Surplus" are the logged share and the logged surplus per recommended item of Section 3.5.

## 5.1 Base models

**Recommender.** On KION the recommender is mid-table (Table 3): level with GRU4Rec, 13% above SASRec and 5% below BERT4Rec for ranked lists, and popularity is a strong floor. Generating a list item by item costs 4% of NDCG@10 against ranking (Section 5.6 shows where). On Yambda the recommender is the best of the trained models, and all of them are far below a replay of the user's own history: NDCG@10 0.187 against 0.117. The split into repeats and new items shows where the trained models stand. Replay scores only on repeats; on tracks the user has not played in the visible history, the recommender is the best ranked model (0.032) and popularity reaches a third of it. Every statement below about relevance on Yambda is relative to this recommender, and the split is reported wherever a method could gain by replaying.

**Reward model.** On held-out events the reward model predicts the consumed share with AUC 0.755 on KION and 0.868 on Yambda (Table 4; $0.7545 \pm 0.0009$ and $0.8668 \pm 0.0006$ over seeds), against 0.656 and 0.607 for the item's mean share and 0.673 and 0.798 for the user's recent mean. The two copies agree (correlation of predictions 0.94 and 0.95). The right half of the table measures what a signal offers a recommender that chooses among its own candidates: how much it varies between the 100 candidates of one user, against how much its mean varies between users. On Yambda the predicted share varies five times less between one user's candidates than between users: it is mostly a property of the listener, which leaves a policy little to act on. On Yambda the like and dislike heads reach AUC 0.74 and 0.73 at base rates of 0.75% and 0.12% of events (Table 30).

## 5.2 How stable the numbers are

Every conclusion of the following sections holds in each of the three seeds (Table 5). The direction of every change of the main run is the same in all seeds, the logged surplus falls by 0.0007 in each of them on KION, and the order of the methods at equal relevance does not change. Three quantities vary more than the rest: relevance at the anchor weight 0.1 on KION, which is near the edge where the policy starts to drift (NDCG@10 $0.079 \pm 0.012$); the personal share of the gain on Yambda ($16\% \pm 16\%$), whose denominator is a gain of one point of share; and the gap between full and myopic credit on KION, about two standard

Table 4: The held-out reward model of the final sessions on test events (AUC; the share heads against the outcome thresholded at one half), and the standard deviation of each prediction between the 100 candidates of one user ("candidates") and of its mean over candidates between users ("users").

| | KION | | | Yambda | | |
|---|---|---|---|---|---|---|
| | **AUC** | **Candidates** | **Users** | **AUC** | **Candidates** | **Users** |
| Consumed share $\hat{\rho}$ | 0.755 | 0.144 | 0.153 | 0.868 | 0.052 | 0.250 |
| Relative share $\hat{q}$ | 0.702 | 0.042 | 0.091 | 0.833 | 0.039 | 0.145 |
| Session continues $\hat{c}$ | 0.657 | 0.050 | 0.093 | 0.603 | 0.006 | 0.016 |
| Return $\hat{u}$ | 0.722 | 0.033 | 0.125 | 0.611 | 0.009 | 0.094 |

Table 5: The central comparisons over three seeds (0, 1, 2): mean ± standard deviation. A seed fixes the initial weights of every model and the sampling of fine-tuning, so it covers the base recommender, both reward models and every run. "Budget" is a KL budget of 0.1 nats on KION and 0.5 on Yambda; "–": not run with three seeds on that dataset.

| | **KION** | **Yambda** |
|---|---|---|
| *Main run ($\beta = 0.3$) against the base recommender* | | |
| NDCG@10, change in % | $-11.5 \pm 1.3$ | $+0.8 \pm 0.7$ |
| Predicted share, held-out model, change in % | $+23.91 \pm 2.09$ | $+1.67 \pm 0.01$ |
| Logged share per recommended item, change in % | $+1.1 \pm 0.4$ | $+1.7 \pm 0.7$ |
| Logged surplus per recommended item, change | $-0.0007 \pm 0.0000$ | $-0.0001 \pm 0.0001$ |
| Personal share of the predicted gain, % | $0.2 \pm 0.5$ | $15.9 \pm 16.3$ |
| KL from the base recommender | $0.091 \pm 0.005$ | $0.038 \pm 0.001$ |
| *What the anchor weight means* | | |
| KL at $\beta = 0.3$, leave-one-out baseline | $0.543 \pm 0.010$ | – |
| Weight holding the KL budget, value model | $0.256 \pm 0.004$ | $0.076 \pm 0.011$ |
| Weight holding the KL budget, leave-one-out | $0.752 \pm 0.017$ | – |
| NDCG@10 at the budget, whole rest of the list | $0.1013 \pm 0.0026$ | $0.0960 \pm 0.0008$ |
| NDCG@10 at the budget, own reward only | $0.1070 \pm 0.0027$ | $0.1076 \pm 0.0014$ |
| NDCG@10 at the budget, one item per episode | $0.1043 \pm 0.0024$ | $0.1075 \pm 0.0038$ |
| NDCG@10 at $\beta = 0.03$ (Yambda's cliff) | – | $0.0061 \pm 0.0043$ |
| *Alternatives and rewards* | | |
| NDCG@10, RL main run | $0.1041 \pm 0.0020$ | $0.1190 \pm 0.0015$ |
| NDCG@10, re-ranking w = 0.3 | $0.1119 \pm 0.0013$ | $0.1155 \pm 0.0008$ |
| Logged share per item, RL main run | $0.0176 \pm 0.0002$ | $0.0689 \pm 0.0008$ |
| Logged share per item, re-ranking w = 0.3 | $0.0201 \pm 0.0001$ | $0.0662 \pm 0.0007$ |
| Personal share of the gain, re-ranking w = 0.3, % | $8.9 \pm 0.3$ | $54.7 \pm 8.8$ |
| NDCG@10, off-policy REINFORCE | $0.1035 \pm 0.0006$ | $0.1256 \pm 0.0009$ |
| NDCG@10, share + 10 × logged relevance, $\beta = 0.3$ | $0.1202 \pm 0.0007$ | – |
| NDCG@10, logged relevance alone, $\beta = 0$ | $0.1179 \pm 0.0004$ | $0.1453 \pm 0.0014$ |
| NDCG@10 on new items, logged relevance alone | – | $0.0198 \pm 0.0005$ |
| *Reference* | | |
| NDCG@10, base recommender | $0.1177 \pm 0.0008$ | $0.1181 \pm 0.0010$ |
| NDCG@10 on new items, base recommender | – | $0.0260 \pm 0.0004$ |
| Reward model AUC, held-out copy | $0.754 \pm 0.001$ | $0.867 \pm 0.001$ |

deviations: crediting each item with its own reward is ahead of full credit in every seed (its lowest NDCG@10, 0.1039, is above the highest of full credit, 0.1037), while one-item episodes overlap with full credit. On Yambda both myopic variants are ahead in every seed. The long-term rewards, the reward models with hidden responses and the analyses by position, user and item come from the review sessions, with seed 0 only.

## 5.3 Q1. How does behaviour change, and why?

**KION: the models agree, the log hardly does.** The training reward model reports a gain of 20.3% and the held-out one 21.5% ($23.9\% \pm 2.1$ over seeds): by the standard check for exploitation the gain is real (Table 6). Relevance falls by 10.7% overall and by 18.2% on skipped titles, and it is kept on liked ones (+1.3%), which looks like a policy that has learned to avoid what users abandon. The other instruments disagree. The policy recommends fewer titles that the user went on to watch (hits −8.3%), and those are finished more often (+9.8%) because they are titles that everybody finishes more often. The two effects almost cancel: the logged share per recommended item rises by 0.6% ($[-0.6, +1.7]$), and by $1.1\% \pm 0.4$ over seeds. Measured against what is usual for each title, users took to the recommended titles less than before: the logged surplus falls from 0.0012 to 0.0005, by 0.0007 in every seed, and the predicted relative share falls too (−1.0%). The same lists shown to *other* users are predicted to earn the same gain (+22.0%).

**Where the impressions go.** The policy moves impressions from series to films: films take 66% of the impressions of the base recommender's generated lists and 80% of the main run's, 89% at $\beta = 0.1$ and all of them without an anchor (Table 7). The five titles that gained most are films of 1.4 to 2.2 hours that viewers usually finish to 0.54–0.69. Four of the five that lost most are series of 9.5 to 22 hours whose usual share, taken of all their episodes, is 0.08 to 0.29, and the fifth is a film usually watched to 0.33 (Table 8). Popularity is not what the policy follows: one of the five largest gains is a film that 953 users watched in the training period, recommended in 17,315 lists instead of 7,308.

Table 6: The main fine-tuning run ($\beta = 0.3$) against the base recommender, seed 0. Changes are relative, in percent, with a 95% paired-bootstrap interval over users where the quantity is defined per user. Table 5 gives the same changes over three seeds.

| | KION | | | Yambda | | |
|---|---|---|---|---|---|---|
| | **before** | **after** | **change, %** | **before** | **after** | **change, %** |
| *Relevance against the user's logged future* | | | | | | |
| NDCG@10, all next items | 0.1173 | 0.1047 | −10.7 [−11.6, −9.8] | 0.1172 | 0.1183 | +0.9 [−0.7, +2.5] |
| NDCG@10, liked items | 0.1214 | 0.1229 | +1.3 [+0.1, +2.4] | 0.1005 | 0.1029 | +2.4 [+0.6, +4.3] |
| NDCG@10, items consumed above usual | 0.1261 | 0.1142 | −9.4 [−10.6, −8.2] | 0.0998 | 0.1023 | +2.5 [+0.7, +4.3] |
| Recall@10, skipped items | 0.1382 | 0.1130 | −18.2 [−19.6, −16.8] | 0.0724 | 0.0735 | +1.5 [−1.7, +4.6] |
| *Predicted consumed share of the list* | | | | | | |
| Training reward model | 0.417 | 0.502 | +20.3 [+20.1, +20.4] | 0.594 | 0.608 | +2.2 [+2.1, +2.4] |
| Held-out reward model | 0.455 | 0.553 | +21.5 [+21.4, +21.7] | 0.599 | 0.609 | +1.7 [+1.5, +1.9] |
| Held-out, list shown to another user | 0.442 | 0.539 | +22.0 [+21.9, +22.2] | 0.498 | 0.508 | +2.0 [+1.7, +2.3] |
| Held-out, relative to the item's usual | 0.529 | 0.524 | −1.0 [−1.1, −1.0] | 0.469 | 0.468 | −0.3 [−0.4, −0.1] |
| *Logged response to the recommended items the user went on to consume* | | | | | | |
| Such items per list | 0.359 | 0.329 | −8.3 | 0.943 | 0.961 | +1.9 |
| Their logged share | 0.481 | 0.528 | +9.8 | 0.710 | 0.713 | +0.5 |
| Logged share per recommended item | 0.0173 | 0.0174 | +0.6 [−0.6, +1.7] | 0.0669 | 0.0686 | +2.5 [+0.3, +4.6] |
| Logged share above the item's usual, per item | 0.0012 | 0.0005 | −61.9 [−69.4, −54.4] | 0.0033 | 0.0032 | −3.0 [−15.2, +9.1] |
| *Lists* | | | | | | |
| Catalogue coverage | 0.495 | 0.449 | −9.2 | 0.519 | 0.506 | −2.6 |
| Overlap between two users' lists | 0.2092 | 0.2509 | +20.0 | 0.0014 | 0.0020 | +42.9 |
| Mean duration of an item, minutes | 339.1 | 205.7 | −39.3 | 3.4 | 3.3 | −3.0 |

Table 7: KION: impressions of films and series in the generated lists of the test requests, for the base recommender and fine-tuned policies (review session).

| | Base recommender | | Fine-tuned | | | |
|---|---|---|---|---|---|---|
| | **Ranked** | **Generated** | **RL, $\beta = 0.3$** | **$\beta = 1$** | **$\beta = 0.1$** | **$\beta = 0$** |
| Films, % of impressions | 72 | 66 | 80 | 70 | 89 | 100 |
| Series, % of impressions | 28 | 34 | 20 | 30 | 11 | 0 |

**One shift per item.** For each history, the change of an item's log-probability under the policy splits into a part that is the same for every history and the rest (Section 3.5). On KION the per-item part explains 78% of the change, and an item's shift correlates 0.90 with its usual consumed share, 0.12 with its log popularity and −0.36 with its duration (Table 9). The policy learned a ranking of titles by how often they are finished and applies it to every user. On Yambda the per-item part explains 44%, and 11% when every item counts alike: the policy moved less (KL 0.04 against 0.09 nats per item), and less of what it changed is one preference per item.

**Why.** The reward makes this the cheapest move. On KION the reward model's prediction of the share varies almost as much between one user's candidates as between users (0.144 against 0.153, Table 4), and most of the variation between candidates is the item: the item's mean share alone predicts the outcome

Table 8: KION: the titles whose number of lists grew and shrank most under the main run. "Viewers in training": users who watched the title in the training period; "usual share": the mean consumed share of the title in the training period; lists among the 59,997 test requests.

| | **Type** | **Hours** | **Viewers in training** | **Usual share** | **Lists before** | **after** |
|---|---|---|---|---|---|---|
| *Gained most* | | | | | | |
| White Snow (2021) | film | 2.2 | 32,435 | 0.64 | 8,388 | 24,472 |
| Palmyra (2020) | film | 1.4 | 18,762 | 0.54 | 8,598 | 21,579 |
| Devyataev (2021) | film | 1.9 | 108,030 | 0.69 | 14,471 | 26,328 |
| ДУХLESS (2011) | film | 1.7 | 953 | 0.55 | 7,308 | 17,315 |
| Masha (2020) | film | 1.5 | 40,254 | 0.63 | 23,485 | 32,316 |
| *Lost most* | | | | | | |
| Themis sees (2019) | series | 16.9 | 25,571 | 0.08 | 33,948 | 10,781 |
| Listened to (2021) | series | 9.5 | 62,651 | 0.18 | 16,278 | 3,343 |
| The Scam (2021) | series | 22.0 | 51,094 | 0.11 | 16,660 | 4,038 |
| The Germans (2021) | series | 10.9 | 33,452 | 0.29 | 14,654 | 5,371 |
| Flash radio (2019) | film | 1.7 | 8,811 | 0.33 | 11,925 | 5,088 |

Table 9: The change of the next-item log-probabilities from the base recommender to a fine-tuned policy, on 2,048 test histories: the share of its variance that one shift per item explains, weighted by the base recommender's probabilities and unweighted, and the correlation of the per-item shift with properties of the item (review sessions).

| | | | Correlation of the item's shift with | | |
|---|---|---|---|---|---|
| | **One shift per item, % of variance** | **unweighted** | **Usual share** | **Log popularity** | **Duration** |
| *KION* | | | | | |
| RL, $\beta = 0.3$ (main run) | 78 | 67 | 0.90 | 0.12 | −0.36 |
| $\beta = 1$ | 70 | 61 | 0.89 | 0.14 | −0.34 |
| $\beta = 0.3$ | 76 | 68 | 0.88 | 0.15 | −0.34 |
| $\beta = 0.1$ | 67 | 70 | 0.75 | 0.21 | −0.29 |
| $\beta = 0$ | 70 | 65 | 0.57 | −0.06 | −0.12 |
| *Yambda* | | | | | |
| RL, $\beta = 0.3$ (main run) | 44 | 11 | 0.47 | 0.04 | −0.21 |
| $\beta = 0.3$ | 45 | 15 | 0.50 | 0.08 | −0.23 |
| $\beta = 0.1$ | 44 | 16 | 0.52 | 0.04 | −0.23 |
| $\beta = 0.05$ | 46 | 21 | 0.53 | 0.09 | −0.20 |

with AUC 0.66. A policy gradient sees one sampled list per history and averages over histories, so the part of the signal that is the same for all of them is what accumulates. The held-out model confirms the gain because it has learned the same item means. The gain is impersonal: 99% of it on KION is what the held-out model predicts for the same lists shown to another user ($0.2\% \pm 0.5\%$ is personal over seeds), and at a weaker anchor the personal share is negative (Table 10). Re-ranking candidates by $\bar{\rho}(j) + w \log \pi_0(j \mid h)$, where $\bar{\rho}(j)$ is the mean consumed share of item $j$ in the training period, uses no reward model and no RL, and reaches the main run's point: a predicted share of 0.549 at NDCG@10 0.109, against 0.553 at 0.105. What RL extracts from the reward model on this reward is, to a first approximation, a ranking of items by how often they are finished.

**Yambda: little to gain.** The same weight moves the policy less than half as far (KL 0.037 against 0.092 nats per item) and changes little: +1.7% of predicted share ($1.67 \pm 0.01$ over seeds), NDCG@10 $+0.8\% \pm 0.7$, and a logged share per recommended item $+1.7\% \pm 0.7$ over seeds (+2.5%, [+0.3, +4.6], in seed 0); the surplus does not move. The policy has shifted slightly towards popular tracks. The value model explains 78% of the variance of returns here, against 49% on KION: the reward is mostly determined by the listener, as Table 4 predicted.

**Answer to Q1.** The policy learns which items everybody finishes, not what each user wants. On KION it moves impressions from long series to short films (films from 66% to 80% of impressions), 78% of its change is one shift per item, and the 24% predicted gain is almost entirely what any other user would also give the same lists; the log confirms 1% of consumed share per recommended item and shows users taking less to the recommended titles than usual. The reason is the reward: the reward model's predictions differ between items far more than between users' reactions to an item, so a ranking of items is what the policy gradient finds. On Yambda the reward depends on the listener, and the policy changes little.

Table 10: The gain in predicted share (held-out model) and the part of it that other users would give the same lists. "Personal" is the share of the gain that depends on who receives the list.

| | **NDCG@10** | **Own user** | **Another user** | **Gain** | **Gain, another** | **Personal** |
|---|---|---|---|---|---|---|
| *KION* | | | | | | |
| Base recommender | 0.1173 | 0.455 | 0.442 | 0.000 | 0.000 | – |
| RL, $\beta = 0.3$ (main run) | 0.1047 | 0.553 | 0.539 | 0.098 | 0.097 | 1% |
| RL, $\beta = 0.1$ | 0.0714 | 0.655 | 0.647 | 0.200 | 0.205 | −3% |
| Re-ranking, reward model, w = 0.3 | 0.1127 | 0.545 | 0.523 | 0.090 | 0.081 | 9% |
| Re-ranking, item mean, w = 0.3 | 0.1088 | 0.549 | 0.535 | 0.094 | 0.093 | 1% |
| *Yambda* | | | | | | |
| Base recommender | 0.1172 | 0.599 | 0.498 | 0.000 | 0.000 | – |
| RL, $\beta = 0.3$ (main run) | 0.1183 | 0.609 | 0.508 | 0.010 | 0.010 | −1% |
| RL, $\beta = 0.1$ | 0.1158 | 0.617 | 0.514 | 0.018 | 0.017 | 7% |
| Re-ranking, reward model, w = 0.3 | 0.1155 | 0.619 | 0.509 | 0.019 | 0.011 | 44% |
| Re-ranking, item mean, w = 0.3 | 0.1136 | 0.621 | 0.518 | 0.021 | 0.020 | 4% |

Table 11: Ranking metrics at $K = 1, 5, 10$ for the main run against the base recommender (generated lists, seed 0). At $K = 1$ NDCG, MAP and HitRate coincide.

| | KION | | | Yambda | | |
|---|---|---|---|---|---|---|
| | **before** | **after** | **change, %** | **before** | **after** | **change, %** |
| *NDCG* | | | | | | |
| NDCG@1 | 0.0885 | 0.0814 | −7.9 | 0.1667 | 0.1655 | −0.7 |
| NDCG@5 | 0.1044 | 0.0920 | −11.9 | 0.1336 | 0.1342 | +0.4 |
| NDCG@10 | 0.1173 | 0.1047 | −10.7 | 0.1172 | 0.1183 | +0.9 |
| *Recall* | | | | | | |
| Recall@1 | 0.0439 | 0.0389 | −11.4 | 0.0159 | 0.0157 | −1.3 |
| Recall@5 | 0.1220 | 0.1050 | −14.0 | 0.0507 | 0.0508 | +0.2 |
| Recall@10 | 0.1620 | 0.1453 | −10.3 | 0.0747 | 0.0756 | +1.2 |
| *MAP* | | | | | | |
| MAP@1 | 0.0885 | 0.0814 | −7.9 | 0.1667 | 0.1655 | −0.7 |
| MAP@5 | 0.0781 | 0.0682 | −12.6 | 0.0997 | 0.0996 | −0.1 |
| MAP@10 | 0.0819 | 0.0719 | −12.2 | 0.0753 | 0.0754 | +0.1 |
| *HitRate* | | | | | | |
| HitRate@1 | 0.0885 | 0.0814 | −7.9 | 0.1667 | 0.1655 | −0.7 |
| HitRate@5 | 0.2224 | 0.2004 | −9.9 | 0.2984 | 0.3051 | +2.2 |
| HitRate@10 | 0.2871 | 0.2645 | −7.9 | 0.3697 | 0.3780 | +2.2 |

## 5.4 Q2. Is relevance kept? Does everyone get the same items?

**Relevance at every cut-off.** On KION every ranking metric falls at every cut-off, by 8 to 14%: NDCG@1 by 7.9%, NDCG@5 by 11.9%, NDCG@10 by 10.7%, Recall@10 by 10.3%, MAP@10 by 12.2% and HitRate@10 by 7.9% (Table 11). The first item is chosen from the real history alone, so the loss at $K = 1$ is the policy's own preference; from the second position on, the generated context adds to it (Section 5.6). Over three seeds NDCG@10 falls by $11.5\% \pm 1.3$. On Yambda every metric stays within 2.2% of the base recommender at every cut-off, and NDCG@10 changes by $+0.8\% \pm 0.7$. By outcome (Table 6), the KION policy keeps relevance on liked titles and loses it on skipped titles (−18.2%) and on titles watched above usual (−9.4%): it has not learned what each user likes, only what everyone finishes.

**Who loses relevance.** On KION the users with the least history lose most (Table 12): users with 5 to 9 events before the request lose 13.5% of NDCG@10 and 2.4% of logged share per item, users with more than 25 events lose 7.4% and gain 3.3% of logged share. For a user the recommender knows little about, the item-level preference of the policy replaces most of what the list had been. On Yambda, where every user has at least 50 events, no group loses.

**Does everyone get the same items?** Not yet at the main run, but closer to it. The base recommender's generated lists on KION are already concentrated: the 1% most shown titles take 72% of the impressions (62% for ranked lists). The main run raises this to 76%, the Gini coefficient of impressions from 0.963 to 0.969, and lowers the catalogue coverage from 0.50 to 0.45 (Table 13). With a weaker anchor the lists converge: 92% at $\beta = 0.1$, and 100% without an anchor, where nearly every user receives the same list (pair overlap 0.98 on KION and 1.00 on Yambda, Table 14). On Yambda the main run moves the share of the most shown 1% from 18% to 20% and the Gini from 0.737 to 0.747; concentration grows markedly only at $\beta = 0.05$ (29%).

Table 12: NDCG@10 and the change of the logged share per recommended item by the user's number of events before the request, in quartiles (review sessions, main run).

| | | NDCG@10 | | | | |
|---|---|---|---|---|---|---|
| | **Users** | **Base, ranked** | **Base, generated** | **RL** | **RL change, %** | **Logged change, %** |
| *KION* | | | | | | |
| 5 to 9 events | 14552 | 0.1537 | 0.1515 | 0.1311 | −13.5 | −2.4 |
| 9 to 14 events | 14121 | 0.1364 | 0.1311 | 0.1171 | −10.7 | +1.2 |
| 14 to 25 events | 15849 | 0.1151 | 0.1097 | 0.0991 | −9.7 | −0.6 |
| 25 to 1143 events | 15475 | 0.0861 | 0.0802 | 0.0743 | −7.4 | +3.3 |
| *Yambda* | | | | | | |
| 50 to 1077 events | 1897 | 0.1164 | 0.1147 | 0.1165 | +1.6 | +1.3 |
| 1077 to 2961 events | 1899 | 0.1179 | 0.1236 | 0.1237 | +0.1 | +3.1 |
| 2961 to 6421 events | 1898 | 0.1110 | 0.1158 | 0.1162 | +0.3 | +2.4 |
| 6421 to 25401 events | 1899 | 0.1101 | 0.1150 | 0.1159 | +0.8 | +1.5 |

Table 13: Concentration of impressions in the lists for the test requests: catalogue coverage, the Gini coefficient of the impressions per item, and the share of impressions taken by the 1% most shown items (review sessions).

| | Coverage | Gini of impressions | Top 1% of items, % of impressions |
|---|---|---|---|
| *KION* | | | |
| Base recommender, ranked | 0.574 | 0.949 | 62 |
| Base recommender, generated | 0.495 | 0.963 | 72 |
| RL, $\beta = 0.3$ (main run) | 0.452 | 0.969 | 76 |
| $\beta = 1$ | 0.477 | 0.966 | 74 |
| $\beta = 0.3$ | 0.433 | 0.972 | 77 |
| $\beta = 0.1$ | 0.278 | 0.990 | 92 |
| $\beta = 0$ | 0.008 | 0.999 | 100 |
| *Yambda* | | | |
| Base recommender, ranked | 0.466 | 0.789 | 23 |
| Base recommender, generated | 0.519 | 0.737 | 18 |
| RL, $\beta = 0.3$ (main run) | 0.512 | 0.747 | 20 |
| $\beta = 0.3$ | 0.510 | 0.749 | 20 |
| $\beta = 0.1$ | 0.492 | 0.766 | 23 |
| $\beta = 0.05$ | 0.446 | 0.808 | 29 |

**What keeps relevance: the distance, not the weight.** Nothing in the reward asks for relevance, so the anchor is what keeps it, and the KL divergence from $\pi_0$ at which a run ends describes how much it keeps (Figure 2). Without an anchor the policy collapses, and the held-out model partly applauds: the training reward model reports a share of 0.84 on KION and the held-out one 0.69, still 51% above the base recommender, while NDCG@10 and the logged share are near zero (Table 14). On KION relevance falls and the predicted share rises steadily as the anchor weakens; on Yambda both move by a few percent down to $\beta = 0.05$, and $\beta = 0.03$ collapses the policy in every seed (NDCG@10 $0.006 \pm 0.004$, KL $3.6 \pm 0.9$), while $\beta = 0.1$ keeps relevance in every seed ($0.1172 \pm 0.0017$).

The weight itself means different things under different estimators. Table 15 changes only the baseline of the policy gradient, with the number of lists per iteration, the learning rate and the passes over a batch held equal. At $\beta = 0.3$ on KION the leave-one-out estimator ends 6.7 times further from the base recommender than the value model (KL $0.543 \pm 0.010$ against $0.081 \pm 0.004$), and at $\beta = 0.1$ it collapses where the value model does not; on Yambda the factor at $\beta = 0.1$ is 15. Under a KL budget of 0.1 the two estimators end at the same distance and on the same frontier (NDCG@10 $0.1037 \pm 0.0021$ against $0.1013 \pm 0.0026$ at predicted shares 0.546 and 0.558), while the weights that hold the budget differ by a factor of 2.9 ($0.752 \pm 0.017$ against $0.256 \pm 0.004$). On Yambda budgets of 0.2 and 0.5 nats end at almost the same weight (0.066 and 0.067): in this region a fixed weight hardly selects an operating point, while a budget does. One reading of the factor is that standardising the advantages gives the policy-gradient term the same scale under both

Table 14: Anchor sweep with the value-model estimator, 1,000 iterations per run. KL is the divergence from the base recommender at the end of fine-tuning, in nats per generated item; "Training model" is the share predicted by the reward model that produced the rewards.

| | KL | NDCG@10 | on liked | Training model | Pred. share | Logged | Coverage | Overlap |
|---|---|---|---|---|---|---|---|---|
| *KION* | | | | | | | | |
| Base recommender | 0.000 | 0.1173 | 0.1214 | 0.417 | 0.455 | 0.0173 | 0.495 | 0.209 |
| $\beta = 1$ | 0.015 | 0.1154 | 0.1245 | 0.445 | 0.487 | 0.0178 | 0.479 | 0.219 |
| $\beta = 0.5$ | 0.040 | 0.1120 | 0.1256 | 0.470 | 0.517 | 0.0180 | 0.458 | 0.237 |
| $\beta = 0.3$ | 0.080 | 0.1043 | 0.1233 | 0.500 | 0.553 | 0.0175 | 0.436 | 0.251 |
| $\beta = 0.2$ | 0.140 | 0.0983 | 0.1193 | 0.521 | 0.576 | 0.0168 | 0.413 | 0.266 |
| $\beta = 0.1$ | 0.590 | 0.0714 | 0.0910 | 0.616 | 0.655 | 0.0114 | 0.287 | 0.316 |
| $\beta = 0$ | 6.215 | 0.0005 | 0.0008 | 0.841 | 0.687 | 0.0002 | 0.008 | 0.983 |
| $\beta = 0.3$, 2,000 iterations | 0.092 | 0.1047 | 0.1229 | 0.502 | 0.553 | 0.0174 | 0.449 | 0.251 |
| *Yambda* | | | | | | | | |
| Base recommender | 0.000 | 0.1172 | 0.1005 | 0.594 | 0.599 | 0.0669 | 0.519 | 0.001 |
| $\beta = 0.3$ | 0.029 | 0.1176 | 0.1018 | 0.604 | 0.608 | 0.0681 | 0.505 | 0.002 |
| $\beta = 0.1$ | 0.102 | 0.1158 | 0.1008 | 0.615 | 0.617 | 0.0668 | 0.479 | 0.004 |
| $\beta = 0.05$ | 0.286 | 0.1115 | 0.0980 | 0.629 | 0.629 | 0.0653 | 0.433 | 0.006 |
| $\beta = 0.03$ | 4.391 | 0.0032 | 0.0033 | 0.871 | 0.774 | 0.0017 | 0.015 | 0.766 |
| $\beta = 0$ | 6.926 | 0.0008 | 0.0009 | 0.861 | 0.810 | 0.0003 | 0.001 | 0.999 |
| $\beta = 0.3$, 2,000 iterations | 0.037 | 0.1183 | 0.1029 | 0.608 | 0.609 | 0.0686 | 0.506 | 0.002 |

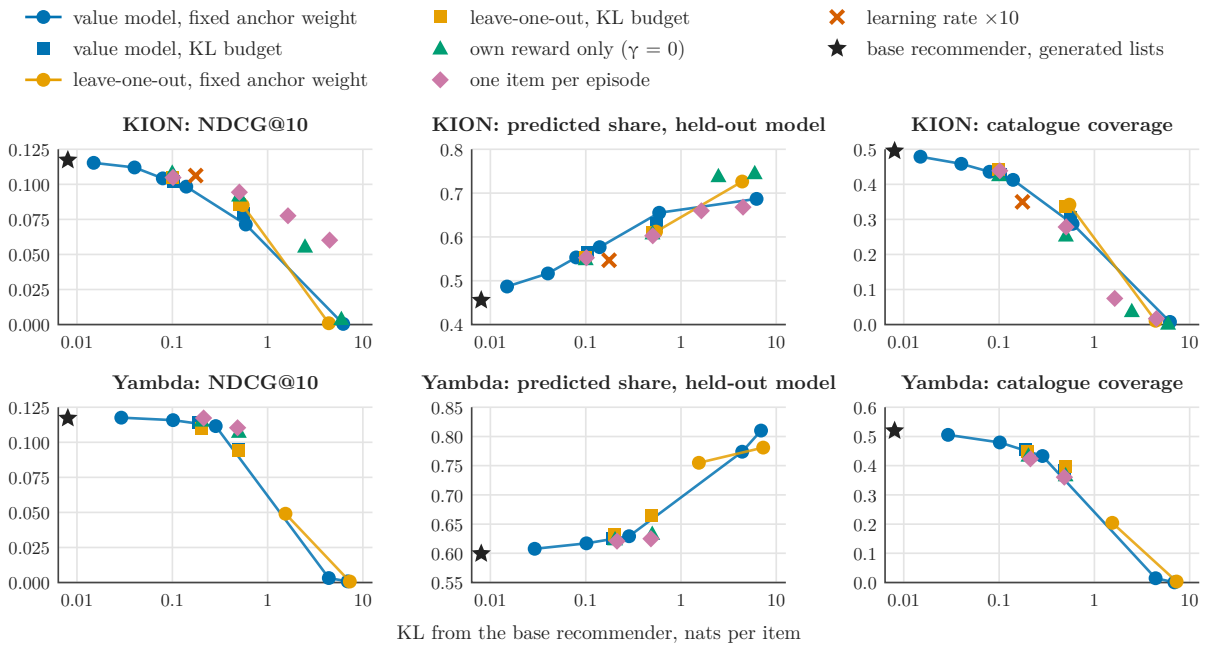



Figure 2: Every fine-tuning run of the default reward against the distance it ended at from the base recommender (logarithmic axis; the base recommender is drawn at the left edge). Runs that differ in the advantage estimator, in how the constraint is set, in credit assignment and in the learning rate lie on one curve. For one-item episodes the distance is that of the first position only.

estimators while the share of that scale that is signal differs; we did not test it. The practical point does not depend on it: an anchor weight is not a quantity that can be compared between estimators, rewards, datasets or papers, and a KL budget is. PPO's clipping touched at most 0.3% of sampled items in every run that did not collapse, the run with ten times the learning rate included.

> **Answer to Q2.** On KION relevance is not fully kept: NDCG@10 falls by $11.5\% \pm 1.3$ at the main operating point, at every cut-off, most for the users with the shortest histories, and the lists concentrate further on the most shown titles (76% of impressions instead of 72%). On Yambda relevance and variety

Table 15: The value model against the leave-one-out baseline at equal anchor weights and at equal KL budgets. "$\beta$ at end" is the weight the run finished with. "Clipped" is the share of sampled items whose probability ratio left PPO's clipping range.

| | β at end | KL | NDCG@10 | Pred. share | Coverage | Clipped |
|---|---|---|---|---|---|---|
| *KION* | | | | | | |
| β = 0.3: value model | 0.300 | 0.080 | 0.1043 | 0.553 | 0.436 | 0.0% |
| β = 0.3: leave-one-out | 0.300 | 0.547 | 0.0852 | 0.612 | 0.342 | 0.0% |
| β = 0.1: value model | 0.100 | 0.590 | 0.0714 | 0.655 | 0.287 | 0.0% |
| β = 0.1: leave-one-out | 0.100 | 4.393 | 0.0009 | 0.726 | 0.011 | 0.0% |
| KL budget 0.1: value model | 0.254 | 0.104 | 0.1018 | 0.564 | 0.427 | 0.0% |
| KL budget 0.1: leave-one-out | 0.740 | 0.099 | 0.1048 | 0.553 | 0.442 | 0.0% |
| KL budget 0.5: value model | 0.112 | 0.556 | 0.0789 | 0.630 | 0.304 | 0.0% |
| KL budget 0.5: leave-one-out | 0.313 | 0.501 | 0.0859 | 0.611 | 0.338 | 0.0% |
| β = 0.3: value model, learning rate ×10 | 0.300 | 0.176 | 0.1062 | 0.547 | 0.350 | 0.3% |
| *Yambda* | | | | | | |
| β = 0.1: value model | 0.100 | 0.102 | 0.1158 | 0.617 | 0.479 | 0.0% |
| β = 0.1: leave-one-out | 0.100 | 1.540 | 0.0491 | 0.755 | 0.204 | 0.0% |
| β = 0.03: value model | 0.030 | 4.391 | 0.0032 | 0.774 | 0.015 | 1.1% |
| β = 0.03: leave-one-out | 0.030 | 7.272 | 0.0007 | 0.781 | 0.003 | 3.3% |
| KL budget 0.2: value model | 0.066 | 0.189 | 0.1138 | 0.625 | 0.454 | 0.0% |
| KL budget 0.2: leave-one-out | 0.215 | 0.200 | 0.1099 | 0.632 | 0.450 | 0.0% |
| KL budget 0.5: value model | 0.067 | 0.490 | 0.0951 | 0.665 | 0.384 | 0.0% |
| KL budget 0.5: leave-one-out | 0.151 | 0.494 | 0.0939 | 0.664 | 0.396 | 0.0% |

Table 16: What a predicted signal offers a recommender that chooses among the 100 best candidates of the base recommender: its standard deviation between one user's candidates, of its mean between users, and the AUC of the signal for telling candidates the user consumed next from the others (held-out reward models of the review sessions).

| | Between candidates | Between users | Relevance AUC |
|---|---|---|---|
| *KION* | | | |
| Consumed share | 0.144 | 0.153 | 0.537 |
| Return | 0.033 | 0.125 | 0.543 |
| *Yambda* | | | |
| Consumed share | 0.059 | 0.241 | 0.532 |
| Return | 0.014 | 0.130 | 0.507 |
| Like | 0.003 | 0.013 | 0.483 |
| Adoption | 0.077 | 0.165 | 0.562 |

are kept. In both logs only the anchor keeps them, the KL distance at which a run ends describes how much, and the anchor weight does not; without the anchor every user receives the same list.

## 5.5 Q3. Long-term goals: return and continued interaction

Four signals describe what follows an item: the session continues, the user returns, the user likes or dislikes the item, and the user adopts it. The first two exist in both logs, the last two only in Yambda. A fifth reward asks for the return after the whole list, paid at its last position (Section 3.3).

**What a policy can choose between.** A reward helps a policy only if it differs between the items the user could be shown. The predicted return varies between one user's candidates a quarter as much as between users on KION (0.033 against 0.125) and a ninth as much on Yambda (0.014 against 0.130); a like varies by 0.003 between candidates (Table 16). Adoption is the exception: it varies between a user's candidates half as much as between users (0.077 against 0.165), relatively more than any other signal, and it is the only one whose ranking of the candidates leans towards the items the user really consumed next (AUC 0.56; the share 0.53, a like 0.48).

**Return per item and continued sessions.** Rewarding the predicted return of each item raises it by 3 to 7% on KION for a loss of 8 to 23% of NDCG@10 and does not move it on Yambda (Table 31). The session signal on KION is derived from calendar days and the order of rows within a day, and we do not interpret its reward.

**Return after the list.** On KION this reward moves the policy further than any per-item reward at the same weight (KL 0.51 at $\beta = 0.3$, against 0.08 for the share). The predicted return after the list rises from 0.73 to 0.86 (+18%), while NDCG@10 falls by 69% and the logged share by 64% (Table 17). The policy found what the reward model associates with an early return, short titles: the mean duration of a

Table 17: Long-term rewards (review sessions, 1,000 iterations). "Δ" is the change of NDCG@10 against the base recommender in percent; "Return" is the held-out model's predicted return after the last item of the list; P(like) and P(adopt) its predicted probability of a like and of adoption, which exist only for Yambda; "Cover." the catalogue coverage; "Hours" the mean duration of a recommended KION title.

| | KL | NDCG@10 | Δ, % | Return | P(like) | P(adopt) | Logged | Cover. | Hours |
|---|---|---|---|---|---|---|---|---|---|
| *KION* | | | | | | | | | |
| Base recommender | 0.000 | 0.1173 | +0.0 | 0.728 | – | – | 0.0173 | 0.495 | 5.7 |
| Return after list, β = 0.3 | 0.507 | 0.0361 | −69.2 | 0.858 | – | – | 0.0062 | 0.539 | 2.2 |
| Return after list, β = 0.1 | 3.423 | 0.0012 | −99.0 | 0.888 | – | – | 0.0002 | 0.059 | 1.7 |
| same, discount 0, β = 0.3 | 0.006 | 0.1144 | −2.5 | 0.733 | – | – | 0.0171 | 0.510 | 5.3 |
| same, discount 0, β = 0.1 | 0.020 | 0.1081 | −7.8 | 0.737 | – | – | 0.0164 | 0.537 | 5.1 |
| *Yambda* | | | | | | | | | |
| Base recommender | 0.000 | 0.1172 | +0.0 | 0.892 | 0.0108 | 0.233 | 0.0669 | 0.519 | – |
| Like − dislike, β = 0.3 | 0.046 | 0.1148 | −2.0 | 0.891 | 0.0117 | 0.234 | 0.0654 | 0.505 | – |
| Like − dislike, β = 0.1 | 0.829 | 0.0841 | −28.2 | 0.878 | 0.0161 | 0.207 | 0.0475 | 0.423 | – |
| Adoption, β = 0.3 | 0.141 | 0.1189 | +1.5 | 0.893 | 0.0121 | 0.277 | 0.0686 | 0.409 | – |
| Adoption, β = 0.1 | 1.204 | 0.0737 | −37.1 | 0.888 | 0.0164 | 0.342 | 0.0407 | 0.129 | – |
| Return after list, β = 0.3 | 0.021 | 0.1170 | −0.2 | 0.892 | 0.0107 | 0.233 | 0.0677 | 0.521 | – |
| Return after list, β = 0.1 | 0.070 | 0.1141 | −2.6 | 0.894 | 0.0106 | 0.230 | 0.0660 | 0.530 | – |
| same, discount 0, β = 0.3 | 0.015 | 0.1171 | −0.1 | 0.892 | 0.0108 | 0.233 | 0.0675 | 0.520 | – |
| same, discount 0, β = 0.1 | 0.029 | 0.1164 | −0.7 | 0.892 | 0.0107 | 0.233 | 0.0675 | 0.521 | – |

recommended title falls from 5.7 to 2.2 hours. Short titles are what the reward model associates with an early return; whether they cause one is what the log cannot say, and a list of ten short films lies far from the lists the model was trained on. With discount 0, which credits only the last item, the policy hardly moves (KL 0.006, return +0.7%): the reward is paid for the list as a whole, and only crediting every item with it reaches it. This is the one reward of the study for which credit along the list matters (Section 5.6), and what it buys is not worth having. On Yambda the return after the list does not move at any weight (0.89).

**Likes.** On Yambda the reward $\hat{l} - \hat{d}$ at $\beta = 0.3$ raises the predicted probability of a like from 1.08% to 1.17% (+8%) for 2% of NDCG@10 (Table 17). At $\beta = 0.1$ the policy moves far (KL 0.83): the probability of a like rises by 49% and that of a dislike by 53%, and NDCG@10 falls by 28%. The policy found tracks that provoke reactions of both kinds, not tracks that are liked more.

**Adoption.** The adoption reward is the one long-term reward that moved without a loss of relevance. At $\beta = 0.3$ the predicted probability that a first play leads to a comeback within 14 days rises from 0.233 to 0.277 (+19%), NDCG@10 by 1.5% and the logged share by 2.5%, at KL 0.14; the price is variety, with catalogue coverage falling from 0.52 to 0.41. At $\beta = 0.1$ the policy drifts (KL 1.2): adoption +47%, NDCG@10 −37%, coverage 0.13. The test week cannot confirm any of it: a first play in the last seven days of the log has no fourteen days after it, so the adoption of the recommended tracks is unobserved.

**What offline evidence cannot settle.** Return, adoption and reactions after a change of policy are counterfactual: the log shows them under the policy that produced it, and the logged check of Section 3.5 covers only items the user reached. Section 6.1 designs the experiment that measures them.

> **Answer to Q3.** Return, likes and adoption can be predicted and rewarded, but at a moderate distance from the base recommender their predictions rise by a few percent. The best case is adoption on Yambda: +19% predicted at no loss of relevance, for a fifth less catalogue coverage. The one large gain, the return after the list on KION (+18%), comes from drifting to short titles at a 69% loss of NDCG@10. Return varies four to nine times less between a user's candidates than between users, so there is little for a policy to choose. None of these gains can be confirmed offline; they need the experiment of Section 6.1.

## 5.6 Q4. Generating several steps ahead

A generated list is built step by step: the item chosen at position $k$ is appended to the history as if the user had consumed it, and the next item is chosen after it. The policy thus plans several steps ahead under an assumption, that each of its items is taken, which is false for most of them: at most 17% of the items at any position are hits. Three questions follow: what the assumption does to the list, what it does to the reward, and whether RL makes use of the steps.

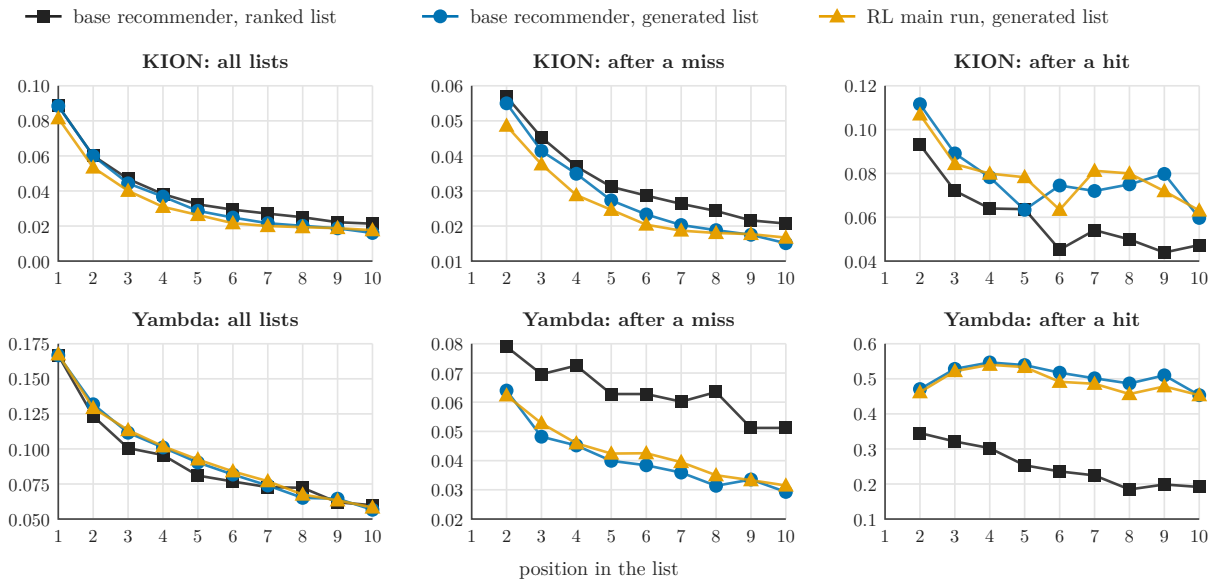



Figure 3: The rate at which the item at each position is among the user's next items, for all lists, after a miss and after a hit at the position before: ranked and generated lists of the base recommender and generated lists of the main run (review sessions; Table 28 has the numbers).

**What generation does to the list.** At position 1 a generated and a ranked list coincide. On KION, from position 3 on, the generated list hits less often than the ranked one (0.0287 against 0.0324 at position 5, 0.0160 against 0.0213 at position 10, Figure 3). The split by the position before explains why. After a miss, the generated list continues from a context that did not happen and hits less often than the ranked list (0.0273 against 0.0312 at position 5); after a hit it continues from a context that did happen and hits more often (0.112 against 0.093 at position 2). On Yambda the effect is large in both directions. After a hit the next generated item is again a hit in about half of the lists (0.54 at position 5) against a quarter for the ranked list (0.25): listening continues in sequences, an album or an artist, which a generated list follows. After a miss the generated list hits at 0.040 against 0.063. Since hits are frequent on Yambda, generation is ahead overall at positions 2 to 7; on KION misses dominate, and generation costs 4% of NDCG@10. Fine-tuning does not change the pattern: the main run's generated lists follow the base recommender's closely after a miss and after a hit.

**What the assumption does to the reward.** To score the $k$-th item the reward model needs the responses to the items before it. On lists whose responses are known, the user's real next five items in order (11,140 KION and 7,153 Yambda requests), we compared the assumptions a simulation can make (Figure 4). With the logged responses fed back, the AUC of the predicted share rises along the list, from 0.757 to 0.782 on KION and from 0.640 to 0.819 on Yambda: the response to the previous item is the best single predictor of the next. Under every assumption the AUC stays near 0.76 on KION and rises only to about 0.69 on Yambda: no simulated response carries that information. The assumptions differ in bias. Feeding back the expected share underestimates the next share on Yambda by 0.06 to 0.09, presumably because the model was rarely shown a response in the middle of a bimodal distribution; sampling a response overestimates slightly (+0.003 to +0.035); assuming that every earlier item was finished, which is what generation assumes about relevance, overestimates by 0.06 to 0.18. The reward model trained with hidden responses and fed "unknown" is nearly unbiased at every later step (+0.001 to +0.013), and it reads observed responses as well as the standard model (AUC 0.755 against 0.758 on KION, 0.867 against 0.868 on Yambda).

**Does the assumption change the policy?** Fine-tuning against the reward model with hidden responses, which is fed "unknown" for generated items, lands on the same frontier as fine-tuning with expected responses: at $\beta = 0.3$ on KION NDCG@10 0.106 against 0.104 at the same distance (KL 0.078 against 0.080)

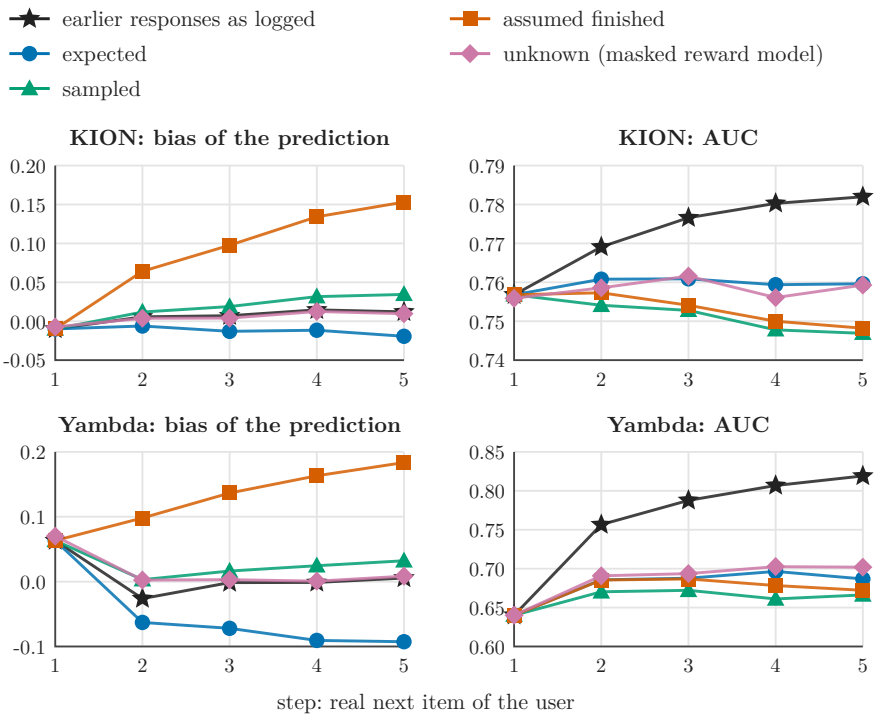



Figure 4: The reward model on the users' real next five items: bias (predicted minus logged share) and AUC of its prediction at each step, when the responses to the earlier items are fed back as logged, as their expectation, as a sample, as finished, or as unknown to a reward model trained with hidden responses (review sessions; Table 29 has the numbers).

Table 18: Credit ablations on the default reward at equal KL budgets and, for KION, at an equal anchor weight (seed 0; Table 5 has three seeds at one budget).

| | β at end | KL | NDCG@10 | on liked | Pred. share | Coverage |
|---|---|---|---|---|---|---|
| *KION, KL budget 0.1* | | | | | | |
| whole rest of the list | 0.254 | 0.104 | 0.1018 | 0.1217 | 0.564 | 0.427 |
| own reward only ($\gamma = 0$) | 1.830 | 0.100 | 0.1084 | 0.1275 | 0.549 | 0.426 |
| one item per episode | 1.597 | 0.102 | 0.1049 | 0.1263 | 0.552 | 0.439 |
| *KION, KL budget 0.5* | | | | | | |
| whole rest of the list | 0.112 | 0.556 | 0.0789 | 0.0993 | 0.630 | 0.304 |
| own reward only ($\gamma = 0$) | 0.760 | 0.502 | 0.0920 | 0.1171 | 0.608 | 0.254 |
| one item per episode | 0.614 | 0.504 | 0.0944 | 0.1208 | 0.603 | 0.279 |
| *KION, fixed $\beta = 0.3$* | | | | | | |
| whole rest of the list | 0.300 | 0.080 | 0.1043 | 0.1233 | 0.553 | 0.436 |
| own reward only ($\gamma = 0$) | 0.300 | 2.469 | 0.0557 | 0.0744 | 0.739 | 0.038 |
| one item per episode | 0.300 | 1.633 | 0.0776 | 0.1026 | 0.660 | 0.075 |
| *Yambda, KL budget 0.2* | | | | | | |
| whole rest of the list | 0.066 | 0.189 | 0.1138 | 0.0993 | 0.625 | 0.454 |
| own reward only ($\gamma = 0$) | 0.267 | 0.203 | 0.1155 | 0.1019 | 0.624 | 0.435 |
| one item per episode | 0.153 | 0.213 | 0.1173 | 0.1033 | 0.621 | 0.423 |
| *Yambda, KL budget 0.5* | | | | | | |
| whole rest of the list | 0.067 | 0.490 | 0.0951 | 0.0836 | 0.665 | 0.384 |
| own reward only ($\gamma = 0$) | 0.165 | 0.499 | 0.1075 | 0.0957 | 0.633 | 0.367 |
| one item per episode | 0.093 | 0.484 | 0.1104 | 0.0980 | 0.625 | 0.361 |

and the same predicted share (0.551 against 0.553); at $\beta = 0.1$ it ends nearer to the base recommender (KL 0.45 against 0.63) and correspondingly more relevant (Table 27). Sampling the response gives the same picture (Table 26). The assumption sets the level of every multi-step predicted reward, a share of 0.55, 0.62 or 0.69 for the same KION policy, but not which policy wins.

**Does RL use the steps?** At an equal KL budget, a policy credited with each item's own reward only ($\gamma = 0$), or trained on one-item episodes, is at least as relevant as one credited with the rest of the list (Table 18). On KION at a budget of 0.1, NDCG@10 is $0.1070 \pm 0.0027$ and $0.1043 \pm 0.0024$ against $0.1013 \pm 0.0026$, at predicted shares 1.4 to 1.6 points lower; on Yambda at a budget of 0.5, $0.1076 \pm 0.0014$ and $0.1075 \pm 0.0038$ against $0.0960 \pm 0.0008$, at shares 3 to 4 points lower. The myopic policies trade a little reward for relevance at the frontier's rate, on it or slightly above it (Figure 2). At a fixed weight they are not comparable: with $\gamma = 0$ the policy ends 31 times further from the base recommender, presumably because per-item rewards standardise to larger advantages than returns that share most of their terms. For rewards that are sums of per-item predictions, crediting an item with what follows it contributed nothing we could detect. The exception is the return after the list (Section 5.5), paid only at the end: there full credit is needed to move at all, and the policy used it to game the reward model.

> **Answer to Q4.** Generating several steps ahead gains after a hit and loses after a miss: on Yambda the next generated item is a hit in 54% of lists after a hit, against 25% in a ranked list, and in 4% against 6% after a miss; on KION misses dominate and generation costs 4% of NDCG@10. The predicted reward of later steps depends on an assumption about responses nobody observed; feeding back expectations or finished items biases it, and a reward model trained with hidden responses is unbiased without losing accuracy. None of the assumptions changes which policy RL finds, and for per-item rewards crediting an item with the rest of the list helps nothing; only a reward paid at the end of the list needed it.

## 5.7 Q5. Trade-off between relevance and reward

**One frontier for all RL settings.** Figure 2 places every fine-tuning run of the default reward at the distance it ended at from the base recommender: different anchor weights, KL budgets, advantage estimators, credit assignments, a learning rate ten times larger, and sampled or hidden simulated responses (Table 27, Table 26). All lie on one curve of relevance, and one of predicted reward, against distance. **RL settings choose a point on the frontier; only the reward function moves the frontier.**

**Equal relevance.** Table 19 reads each method's curve at a fixed loss of relevance: the largest predicted gain it reaches while staying within 1% or 5% of the base recommender's NDCG@10, and the logged share

Table 19: Methods compared at equal relevance (final sessions, seed 0): the largest change of the predicted share (held-out model) that a method reaches without losing more than 1% or 5% of the base recommender's NDCG@10, and the change of the logged share per recommended item at that point, in percent against the base recommender's generated lists. Between two runs of a method the curve is the straight line that a mixture of the two policies achieves. "–": the method does not reach that relevance.

| | **NDCG@10 at most 1% below the base** | | **NDCG@10 at most 5% below the base** | |
|---|---|---|---|---|
| | **Predicted share, %** | **Logged share, %** | **Predicted share, %** | **Logged share, %** |
| *KION* | | | | |
| Ranked list of the base recommender | +2.4 | +11.7 | +2.4 | +11.7 |
| RL on the share, anchor sweep | +4.2 | +1.8 | +14.1 | +3.7 |
| RL on share + 10 × logged relevance | +16.9 | +8.8 | +23.1 | +6.7 |
| Re-ranking by the reward model | +14.9 | +15.6 | +20.7 | +15.5 |
| Off-policy REINFORCE | +1.4 | +0.3 | +6.9 | +1.4 |
| Liked-only fine-tuning | +1.7 | +0.3 | +8.4 | +1.4 |
| *Yambda* | | | | |
| Ranked list of the base recommender | +0.6 | −1.2 | +1.9 | −3.6 |
| RL on the share, anchor sweep | +2.8 | +0.1 | +5.0 | −2.5 |
| RL on share + 10 × logged relevance | +1.4 | +24.1 | +1.4 | +24.1 |
| Re-ranking by the reward model | – | – | +5.3 | −3.8 |
| Off-policy REINFORCE | +1.9 | +9.9 | +1.9 | +9.9 |
| Liked-only fine-tuning | +1.2 | +3.4 | +1.2 | +3.4 |

at that point. Between two runs of a method the curve is taken to be the straight line, which is exactly what a policy achieves that serves one run's lists to a random part of the users and the other run's to the rest. On KION, RL on the share gains 4.2% of predicted share within 1% of relevance and 14.1% within 5%, and 1.8% and 3.7% of logged share. Adding ten times the logged-relevance term to the reward moves the frontier: 16.9% and 23.1% of predicted share, 8.8% and 6.7% of logged share. Re-ranking by the same reward model reaches 14.9% and 20.7%, with the largest logged gains (15.6%), of which the ranked list it starts from contributes 11.7% with no reward at all. Off-policy REINFORCE and liked-only fine-tuning, which never consult the reward model, gain least in predicted share. On Yambda the differences at 1% are within the noise between seeds (the standard deviation of the base recommender's NDCG@10 over seeds is 0.9% of it), and the methods that learn from the log lead in logged share: +24% for logged relevance and +10% for off-policy REINFORCE, through repeats (Table 33).

**Does it take RL?** At nearly the same predicted share as the main RL run, re-ranking with $w = 0.3$ is more relevant (NDCG@10 $0.1119 \pm 0.0013$ against $0.1041 \pm 0.0020$ over seeds), has a higher logged share per recommended item ($0.0201 \pm 0.0001$ against $0.0176 \pm 0.0002$), covers more of the catalogue, and $9.0\% \pm 0.3$ of its gain is personal against $0.2\% \pm 0.5$ for RL (Table 20, Figure 5). Most of the difference in relevance is a head start: re-ranking begins from the ranked lists of the base recommender and RL from its generated lists, which are 4% less relevant. Measured from its own starting point, each method pays about the same relevance for the same predicted share (−7.6% of NDCG@10 for +0.08 of share by re-ranking, −10.7% for +0.10 by RL). The logged share is the axis on which re-ranking moves and RL does not. A reward for the whole list, under which an item's worth depends on its neighbours, does not change the picture: greedy re-ranking reaches a higher list reward than RL at less cost in relevance (Table 35). On Yambda off-policy REINFORCE on logged sessions raises NDCG@10 by 6% ($0.1256 \pm 0.0009$) and the logged share by 10%, and re-ranking is less relevant than RL ($0.1155 \pm 0.0008$ against $0.1190 \pm 0.0015$), because the ranked lists it starts from are less relevant than the generated ones there.

**Rewards that move the frontier.** Logged relevance is the reward term that moves it upwards. On KION the share plus ten times logged relevance at $\beta = 0.3$ improves on the base recommender in relevance (NDCG@10 $0.1202 \pm 0.0007$ against $0.1177 \pm 0.0008$ over seeds), predicted share (+10%) and logged share (+7%) at once; without the anchor its coverage falls from 0.50 to 0.11 (Table 32). On Yambda logged relevance alone, with no anchor, raises NDCG@10 by 23% ($0.1453 \pm 0.0014$), the largest gain in relevance of the study, by replaying the user's history: relevance on new tracks falls from 0.0260 to 0.0198 and the share of recommendations taken from the history rises from 51% to 62% (Table 33). It is a legitimate gain

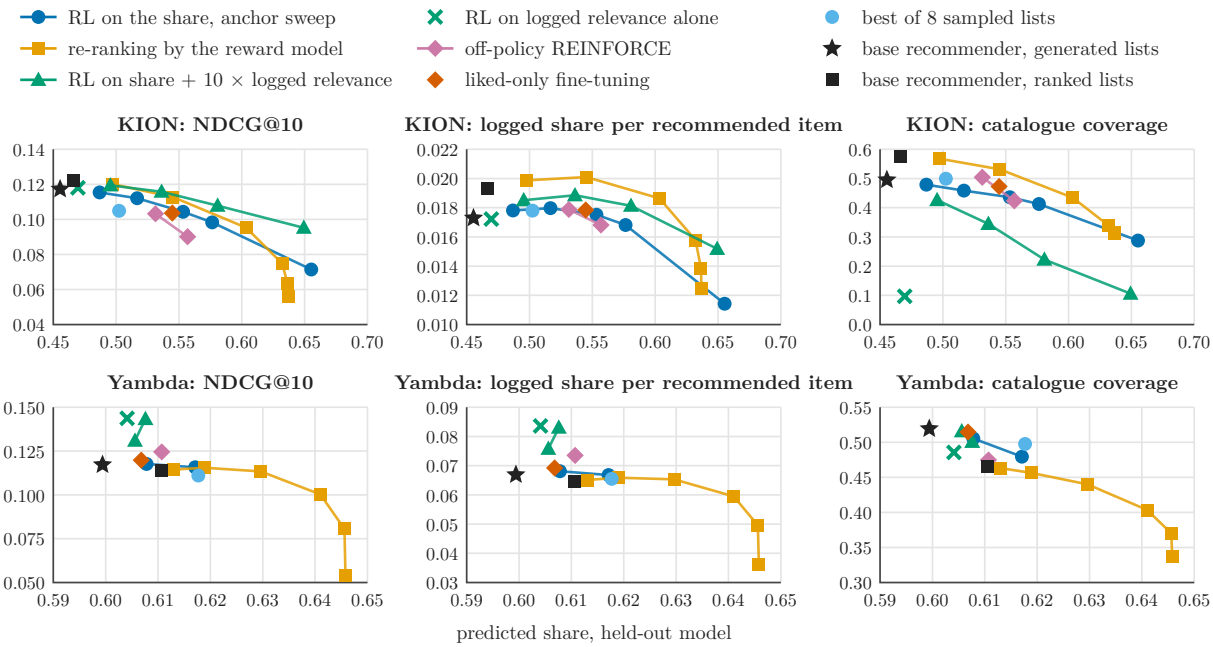



Figure 5: Methods compared at the predicted share they reach (held-out model): relevance, the logged share per recommended item, and coverage. Points of a line are one method at several strengths. RL and the fine-tuning methods start from the generated lists of the base recommender, re-ranking from its ranked lists.

on this metric and not better recommendation of new music. A reward neutral to item length, the relative share, moves the frontier sideways: the drift towards films disappears (the mean duration of a recommended title rises from 5.7 to 6.7 hours where the share reward took it to 3.4), relevance is kept ($-0.4\%$ at $\beta = 0.3$), and almost nothing is gained, because the part of the reward that depends on the user is small (Table 21). A reward that subtracts what the same model predicts for another user cannot collapse onto one list even without an anchor (pair overlap 0.33 against 0.98 on KION), and raises the personal part slightly (Table 34).

**Answer to Q5.** RL settings choose a point on the frontier; only the reward function moves the frontier. The anchor weight, the KL budget, the estimator, the credit assignment and the simulated responses all move the policy along one curve of relevance against reward. Within 1% of the base recommender's

Table 20: Ways of raising the reward without on-policy RL. Re-ranking starts from the ranked lists of the base recommender, every other method from its generated lists.

| | NDCG@10 | on liked | above usual | Pred. share | Logged | Surplus | Coverage |
|---|---|---|---|---|---|---|---|
| *KION* | | | | | | | |
| Base recommender | 0.1173 | 0.1214 | 0.1261 | 0.455 | 0.0173 | 0.0012 | 0.495 |
| Base recommender, ranked | 0.1220 | 0.1293 | 0.1326 | 0.466 | 0.0193 | 0.0014 | 0.574 |
| RL, $\beta = 0.3$ (main run) | 0.1047 | 0.1229 | 0.1142 | 0.553 | 0.0174 | 0.0005 | 0.449 |
| Liked-only fine-tuning | 0.1036 | 0.1258 | 0.1134 | 0.545 | 0.0178 | 0.0003 | 0.473 |
| Off-policy REINFORCE, $\varkappa = 0.5$ | 0.1032 | 0.1237 | 0.1127 | 0.531 | 0.0179 | 0.0005 | 0.504 |
| Re-ranking, w = 1 | 0.1201 | 0.1334 | 0.1315 | 0.497 | 0.0199 | 0.0013 | 0.568 |
| Re-ranking, w = 0.3 | 0.1127 | 0.1344 | 0.1249 | 0.545 | 0.0201 | 0.0009 | 0.531 |
| Re-ranking, w = 0.1 | 0.0953 | 0.1234 | 0.1080 | 0.603 | 0.0186 | 0.0003 | 0.434 |
| Best of 8 sampled lists | 0.1049 | 0.1125 | 0.1128 | 0.502 | 0.0178 | 0.0010 | 0.499 |
| *Yambda* | | | | | | | |
| Base recommender | 0.1172 | 0.1005 | 0.0998 | 0.599 | 0.0669 | 0.0033 | 0.519 |
| Base recommender, ranked | 0.1138 | 0.0989 | 0.0986 | 0.611 | 0.0645 | 0.0029 | 0.466 |
| RL, $\beta = 0.3$ (main run) | 0.1183 | 0.1029 | 0.1023 | 0.609 | 0.0686 | 0.0032 | 0.506 |
| Liked-only fine-tuning | 0.1198 | 0.1035 | 0.1028 | 0.607 | 0.0692 | 0.0036 | 0.515 |
| Off-policy REINFORCE, $\varkappa = 0.5$ | 0.1245 | 0.1090 | 0.1083 | 0.611 | 0.0735 | 0.0039 | 0.475 |
| Re-ranking, w = 1 | 0.1144 | 0.0998 | 0.0995 | 0.613 | 0.0651 | 0.0030 | 0.463 |
| Re-ranking, w = 0.3 | 0.1155 | 0.1012 | 0.1008 | 0.619 | 0.0659 | 0.0029 | 0.456 |
| Re-ranking, w = 0.1 | 0.1134 | 0.0998 | 0.0993 | 0.630 | 0.0653 | 0.0028 | 0.440 |
| Best of 8 sampled lists | 0.1110 | 0.0967 | 0.0961 | 0.618 | 0.0655 | 0.0030 | 0.498 |

Table 21: Rewards that an item cannot earn on its own account, and logged relevance. "Rel. share" is the held-out model's prediction of the share relative to the item's usual (Equation 1); 0.5 is "as usual".

| | NDCG@10 | on liked | Pred. share | Rel. share | Logged | Surplus | Coverage |
|---|---|---|---|---|---|---|---|
| *KION* | | | | | | | |
| Base recommender | 0.1173 | 0.1214 | 0.455 | 0.529 | 0.0173 | 0.0012 | 0.495 |
| Share (the main reward), $\beta = 0.3$ | 0.1043 | 0.1233 | 0.553 | 0.525 | 0.0175 | 0.0005 | 0.436 |
| Base recommender, ranked | 0.1220 | 0.1293 | 0.466 | 0.528 | 0.0193 | 0.0014 | 0.574 |
| Relative share, $\beta = 0.3$ | 0.1168 | 0.1190 | 0.434 | 0.541 | 0.0168 | 0.0013 | 0.481 |
| Relative share, $\beta = 0.1$ | 0.1114 | 0.1111 | 0.401 | 0.554 | 0.0154 | 0.0014 | 0.439 |
| Re-ranking by relative share, w = 0.1 | 0.1206 | 0.1263 | 0.437 | 0.546 | 0.0185 | 0.0020 | 0.552 |
| Share + 10 × logged relevance, $\beta = 0.3$ | 0.1194 | 0.1303 | 0.495 | 0.535 | 0.0185 | 0.0011 | 0.426 |
| Share + 10 × logged relevance, $\beta = 0.1$ | 0.1158 | 0.1323 | 0.536 | 0.540 | 0.0188 | 0.0008 | 0.345 |
| Logged relevance alone, $\beta = 0$ | 0.1180 | 0.1279 | 0.470 | 0.553 | 0.0172 | 0.0007 | 0.097 |
| *Yambda* | | | | | | | |
| Base recommender | 0.1172 | 0.1005 | 0.599 | 0.469 | 0.0669 | 0.0033 | 0.519 |
| Share (the main reward), $\beta = 0.1$ | 0.1158 | 0.1008 | 0.617 | 0.465 | 0.0668 | 0.0029 | 0.479 |
| Base recommender, ranked | 0.1138 | 0.0989 | 0.611 | 0.473 | 0.0645 | 0.0029 | 0.466 |
| Relative share, $\beta = 0.1$ | 0.1107 | 0.0960 | 0.595 | 0.487 | 0.0628 | 0.0040 | 0.476 |
| Re-ranking by relative share, w = 0.1 | 0.1120 | 0.0977 | 0.610 | 0.486 | 0.0633 | 0.0034 | 0.444 |
| Share + 10 × logged relevance, $\beta = 0.1$ | 0.1309 | 0.1139 | 0.606 | 0.468 | 0.0758 | 0.0035 | 0.516 |
| Share + 10 × logged relevance, $\beta = 0$ | 0.1433 | 0.1245 | 0.608 | 0.467 | 0.0830 | 0.0037 | 0.501 |
| Logged relevance alone, $\beta = 0$ | 0.1437 | 0.1259 | 0.604 | 0.468 | 0.0836 | 0.0039 | 0.486 |

NDCG@10, RL on the share gains 4% of predicted share on KION, the same RL with logged relevance in the reward 17%, and re-ranking by the same reward model, which needs no training, 15%, with the largest logged gain. Logged relevance is the reward term that improves logged outcomes, and on Yambda it does so by replaying the user's history.

# 6 Discussion

**What the instruments add.** The usual defence against reward hacking is a second model, and here it fails in an instructive way: it catches the exploitation that is specific to one set of weights (0.84 against 0.69 without an anchor) and misses the exploitation that any model of this kind invites. Both reward models have learned, correctly, that some titles are finished more often than others, and a policy that serves those titles to everyone raises both. The instruments that expose this are cheap: score the same lists for another user, split the policy's change into a part per item and the rest, and count what the log says about the recommended items the user reached.

**What RL added.** For a reward that is a sum of per-item predictions and is mostly determined by the item, the optimum of the anchored objective is close to a re-ranking of $\pi_0$ by item quality, and a policy gradient is an expensive way to approximate it: a fine-tuning run costs minutes of GPU time per operating point, a re-ranking weight costs nothing. The estimator comparison and the credit ablations say the same from the inside: at an equal distance from $\pi_0$, nothing that distinguishes one policy-gradient method from another mattered. The sequential side of RL mattered once, for a reward paid after the whole list, and there it served to exploit the reward model.

**Relation to the studies we build on.** Petrov and Macdonald [8] keep relevance inside the reward; our results agree that this term is what keeps hits, and add that it keeps whatever is easiest to hit, so that an anchor or a diversity term has to keep the rest. Matveev et al. [9] learn from logged actions and logged feedback; on the log with a recommender flag and long sessions this was the best method in logged terms, and their warning about judging with the model that selected applies to our predicted shares in full.

## 6.1 An online experiment

The claims that the offline evidence leaves open (Table 22) are about responses to lists the log never showed. They need an experiment that serves the lists.

- **Arms.** (A) The base recommender's generated lists. (B) The RL policy at the operating point selected on validation requests ($\beta = 1$ on KION, $\beta = 0.05$ on Yambda). (C) Re-ranking by the reward model at the predicted share of B. (D) On the music service, RL on the adoption reward at $\beta = 0.3$. B against

Table 22: What the offline evidence settles and what needs an online experiment.

| Claim | Offline evidence here | What an online experiment would add |
|---|---|---|
| Without an anchor the lists collapse | Conclusive: a fact about the policy, measured without any model of the user | Nothing |
| An anchor weight is not comparable, a KL distance is | Conclusive for this setup, in three seeds: a statement about the optimiser | Nothing |
| The predicted gain is impersonal | Conclusive as a statement about the reward model's predictions | Whether real responses to the fine-tuned lists depend on the user more than predicted |
| The predicted gain on KION is not a gain in consumed share | Suggestive: the logged share per recommended item rises by 1%, but only for the items users reached on their own | The consumed share of served recommendations, the decisive measurement |
| Users with short histories lose most | Measured on logged futures | Whether they also consume less |
| Re-ranking is at least as good as RL | Under the same reward model and the same logged checks | Real responses; serving cost of scoring 100 candidates against one generation pass |
| Long-term rewards buy little; adoption is the exception | Weak: return and adoption are counterfactual, and adoption is not observable in a test week | Return and adoption rates under each policy |
| Generated lists gain after a hit and lose after a miss | Conclusive for logged futures | Whether users who skip an item are served worse lists in practice |

A measures what fine-tuning does to users; C against B whether RL is needed; D against A whether a long-term reward is real.

- **Unit and assignment.** Users, assigned by a hash of the user identifier, so that each user sees one policy throughout; new users are assigned on arrival.
- **Duration.** Four weeks: one week for exposure to settle, then fourteen days of adoption window for the first plays of the second week, and one more week for returns.
- **Primary metrics.** Per user: the consumed share of the recommended items that were served, which decides Q1; the return rate within the return window (a week on KION, a day on Yambda); on Yambda the share of first plays of recommended tracks that are played again in a later session within 14 days.
- **Guardrails.** Recommendation-driven consumption per user, likes and dislikes per recommended item, catalogue coverage and the Gini coefficient of impressions, the share of new items, and every metric for the quartile of users with the shortest histories, who lose most offline (Table 12).
- **Analysis.** Differences of per-user means with a CUPED adjustment on the same metric in the four weeks before the experiment, intervals by a bootstrap over users, and the history quartiles as pre-registered segments. B is better than A if the consumed share of served recommendations rises and the return rate does not fall; D is better than A if adoption rises and the consumed share does not fall.
- **Size.** For a per-user binary outcome with a base rate near 0.7, a relative change of 1% (0.7 points) needs about $2(1.96 + 0.84)^2 \cdot 0.7 \cdot 0.3/0.007^2 \approx 67,000$ users per arm (two-sided $\alpha = 0.05$, power 0.8). The KION test week alone has 60,000 users with a request.

**Practical implications.** Report the KL distance a fine-tuned policy ended at, not only the weight that was set. Before fine-tuning against a feedback model, re-rank by the same model and by item means: if the frontier is the same, the model's contribution is knowledge about items. Score lists for another user, and decompose the policy's change into a part per item. Prefer a reward that an item cannot earn on its own account, and expect it to be worth less than the raw signal suggested. Simulate generated items with a reward model trained to read unknown responses.

# 7 Limitations

- **Three seeds, for the core only.** The central comparisons have three seeds; the long-term rewards, the reward models with hidden responses and the analyses by position, user and item have one. A standard deviation over three seeds is itself uncertain.
- **Everything is offline.** Table 22 lists what this leaves open, and Section 6.1 how to close it. The logged check sees only recommended items that users reached without the recommender's help.

- **Adoption is not verified.** Its label needs fourteen days after a first play, which the test week does not have. Training-period first plays of held-out users, or a longer test period, would allow a check.
- **The Yambda grids are narrower than KION's.** Sampled feedback, the list reward, and the retention and personal rewards were run at one anchor weight each.
- **The base recommender on Yambda is below a replay heuristic,** and gains in relevance there are gains over this recommender.
- **The main run's weight is not a blind choice** (Section 4).
- **The explanation of why weights differ between estimators is a hypothesis** that no run isolates.
- **The personal part is the reward model's own belief** about how much of its prediction depends on the user.
- **Re-ranking and RL start from different lists,** ranked and generated; on KION this favours re-ranking by about 4%.
- **Equal-relevance curves interpolate between runs.** The straight line is achievable by mixing two policies, but a method may reach more between two of its runs than the line shows.
- **Baselines are untuned.**

# 8 Conclusion

We fine-tuned a generative recommender against a learned reward model in the setting with the fewest protections, on a video and a music log, with three seeds for every central comparison, and answered five questions.

**Q1. How does behaviour change, and why?** The policy learned which items everybody finishes and serves them to everyone; the reward model's predictions differ between items more than between users, and a ranking of items is what a policy gradient finds in them. **Q2. Is relevance kept? Does everyone get the same items?** On KION relevance falls by a tenth, most for the users with the least history, and impressions concentrate further; on Yambda both are kept; the KL distance decides, and without an anchor everyone receives the same list. **Q3. Long-term goals.** Rewards for return, likes and adoption buy a few percent of their predictions at a moderate distance; the large gain of a reward paid after the list was reward hacking, and none of it can be confirmed offline. **Q4. Generating several steps ahead.** A generated list gains after a hit and loses after a miss; simulated responses bias the reward of later steps unless the reward model was trained to read unknown ones, and crediting items with the rest of the list helps no per-item reward. **Q5. Trade-off between relevance and reward.** RL settings choose a point on the frontier; only the reward function moves the frontier, and re-ranking by the same reward model reaches more reward at equal relevance than RL.

What moved logged outcomes came from the log: a logged-relevance term inside the reward, and off-policy learning from logged sessions. For a reward model to be worth optimising with RL, it has to know something about the user that cannot be looked up per item, and the reward has to be built so that only that knowledge pays. In our data that part was small. Whether real users respond to the fine-tuned lists as the log suggests is the question of the experiment in Section 6.1.

## Code and data availability

The code, the configuration and every table of every session are in the repository `github.com/daniilantonovv/RecSysMaxxing` (`results/`); the tables of this paper are typeset from those files by `report/tables.py` and the SVG figures by `report/figures.py`.

# References


[1] W.-C. Kang and J. McAuley, "Self-Attentive Sequential Recommendation," *arXiv preprint arXiv:1808.09781*, 2018.

[2] A. V. Petrov and C. Macdonald, "Generative Sequential Recommendation with GPTRec," *arXiv preprint arXiv:2306.11114*, 2023.

[3] S. Rajput *et al.*, "Recommender Systems with Generative Retrieval," *arXiv preprint arXiv:2305.05065*, 2023.

[4] J. Zhai *et al.*, "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations," *arXiv preprint arXiv:2402.17152*, 2024.

[5] J. Deng *et al.*, "OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment," *arXiv preprint arXiv:2502.18965*, 2025.

[6] M. Chen, A. Beutel, P. Covington, S. Jain, F. Belletti, and E. Chi, "Top-K Off-Policy Correction for a REINFORCE Recommender System," *arXiv preprint arXiv:1812.02353*, 2018.

[7] E. Ie *et al.*, "Reinforcement Learning for Slate-based Recommender Systems: A Tractable Decomposition and Practical Methodology," *arXiv preprint arXiv:1905.12767*, 2019.

[8] A. Petrov and C. Macdonald, "Aligning GPTRec with Beyond-Accuracy Goals with Reinforcement Learning," *arXiv preprint arXiv:2403.04875*, 2024.

[9] A. Matveev, S. Makeev, A. Krasilnikov, V. Baikalov, S. Liamaev, and K. Khrylchenko, "Session-Level Optimization for Large-Scale Retrieval using REINFORCE with Multi-Step Off-Policy Correction," *arXiv preprint arXiv:2607.02818*, 2026.

[10] T. Yu *et al.*, "MOPO: Model-based Offline Policy Optimization," *arXiv preprint arXiv:2005.13239*, 2020.

[11] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "MOReL: Model-Based Offline Reinforcement Learning," *arXiv preprint arXiv:2005.05951*, 2020.

[12] L. Gao, J. Schulman, and J. Hilton, "Scaling Laws for Reward Model Overoptimization," *arXiv preprint arXiv:2210.10760*, 2022.

[13] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger, "Defining and Characterizing Reward Hacking," *arXiv preprint arXiv:2209.13085*, 2022.

[14] C. Gao *et al.*, "Alleviating Matthew Effect of Offline Reinforcement Learning in Interactive Recommendation," *arXiv preprint arXiv:2307.04571*, 2023.

[15] C. Gao *et al.*, "CIRS: Bursting Filter Bubbles by Counterfactual Interactive Recommender System," *arXiv preprint arXiv:2204.01266*, 2022.

[16] R. Jiang, S. Chiappa, T. Lattimore, A. György, and P. Kohli, "Degenerate Feedback Loops in Recommender Systems," *arXiv preprint arXiv:1902.10730*, 2019.

[17] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019.

[18] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based Recommendations with Recurrent Neural Networks," *arXiv preprint arXiv:1511.06939*, 2015.

[19] F. Sun *et al.*, "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer," *arXiv preprint arXiv:1904.06690*, 2019.

[20] E. Botta *et al.*, "PinRec: Unified Generative Retrieval for Pinterest Recommender Systems," *arXiv preprint arXiv:2504.10507*, 2025.

[21] R. Deffayet, T. Thonet, J.-M. Renders, and M. de Rijke, "Generative Slate Recommendation with Reinforcement Learning," *arXiv preprint arXiv:2301.08632*, 2023.

[22] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Self-Supervised Reinforcement Learning for Recommender Systems," *arXiv preprint arXiv:2006.05779*, 2020.

[23] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Supervised Advantage Actor-Critic for Recommender Systems," *arXiv preprint arXiv:2111.03474*, 2021.

[24] K. Christakopoulou *et al.*, "Reward Shaping for User Satisfaction in a REINFORCE Recommender," *arXiv preprint arXiv:2209.15166*, 2022.

[25] E. Ie *et al.*, "RecSim: A Configurable Simulation Platform for Recommender Systems," *arXiv preprint arXiv:1909.04847*, 2019.

[26] J.-C. Shi, Y. Yu, Q. Da, S.-Y. Chen, and A.-X. Zeng, "Virtual-Taobao: Virtualizing Real-world Online Retail Environment for Reinforcement Learning," *arXiv preprint arXiv:1805.10000*, 2018.

[27] K. Zhao *et al.*, "KuaiSim: A Comprehensive Simulator for Recommender Systems," *arXiv preprint arXiv:2309.12645*, 2023.

[28] M. Janner, J. Fu, M. Zhang, and S. Levine, "When to Trust Your Model: Model-Based Policy Optimization," *arXiv preprint arXiv:1906.08253*, 2019.

[29] T. Coste, U. Anwar, R. Kirk, and D. Krueger, "Reward Model Ensembles Help Mitigate Overoptimization," *arXiv preprint arXiv:2310.02743*, 2023.

[30] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust Region Policy Optimization," *arXiv preprint arXiv:1502.05477*, 2015.

[31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[32] N. Jaques, S. Gu, D. Bahdanau, J. M. Hernández-Lobato, R. E. Turner, and D. Eck, "Sequence Tutor: Conservative Fine-Tuning of Sequence Generation Models with KL-control," *arXiv preprint arXiv:1611.02796*, 2016.

[33] N. Jaques *et al.*, "Way Off-Policy Batch Deep Reinforcement Learning of Implicit Human Preferences in Dialog," *arXiv preprint arXiv:1907.00456*, 2019.

[34] D. M. Ziegler *et al.*, "Fine-Tuning Language Models from Human Preferences," *arXiv preprint arXiv:1909.08593*, 2019.

[35] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *arXiv preprint arXiv:2203.02155*, 2022.

[36] Y. Wu, G. Tucker, and O. Nachum, "Behavior Regularized Offline Reinforcement Learning," *arXiv preprint arXiv:1911.11361*, 2019.

[37] R. Zhan *et al.*, "Deconfounding Duration Bias in Watch-time Prediction for Video Recommendation," *arXiv preprint arXiv:2206.06003*, 2022.

[38] H. Zhao, L. Zhang, J. Xu, G. Cai, Z. Dong, and J.-R. Wen, "Uncovering User Interest from Biased and Noised Watch Time in Video Recommendation," *arXiv preprint arXiv:2308.08120*, 2023.

[39] L. Zou, L. Xia, Z. Ding, J. Song, W. Liu, and D. Yin, "Reinforcement Learning to Optimize Long-term User Engagement in Recommender Systems," *arXiv preprint arXiv:1902.05570*, 2019.

[40] Q. Cai *et al.*, "Reinforcing User Retention in a Billion Scale Short Video Recommender System," *arXiv preprint arXiv:2302.01724*, 2023.

[41] T. A. Mann *et al.*, "Learning from Delayed Outcomes via Proxies with Applications to Recommender Systems," *arXiv preprint arXiv:1807.09387*, 2018.

[42] Y. Su *et al.*, "Long-Term Value of Exploration: Measurements, Findings and Algorithms," *arXiv preprint arXiv:2305.07764*, 2023.

[43] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336.

[44] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks," *arXiv preprint arXiv:1506.03099*, 2015.

[45] M. Ranzato, S. Chopra, M. Auli, and W. Zaremba, "Sequence Level Training with Recurrent Neural Networks," *arXiv preprint arXiv:1511.06732*, 2015.

[46] C. Guo *et al.*, "PROMISE: Process Reward Models Unlock Test-Time Scaling Laws in Generative Recommendations," *arXiv preprint arXiv:2601.04674*, 2026.

[47] R. Deffayet, T. Thonet, J.-M. Renders, and M. de Rijke, "Offline Evaluation for Reinforcement Learning-based Recommendation: A Critical Issue and Some Alternatives," *arXiv preprint arXiv:2301.00993*, 2023.

[48] M. Dudik, J. Langford, and L. Li, "Doubly Robust Policy Evaluation and Learning," *arXiv preprint arXiv:1103.4601*, 2011.

[49] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches," *arXiv preprint arXiv:1907.06902*, 2019.

[50] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[51] R. J. Williams, "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.

[52] W. Kool, H. van Hoof, and M. Welling, "Buy 4 REINFORCE Samples, Get a Baseline for Free!," in *ICLR 2019 Workshop on Deep Reinforcement Learning Meets Structured Prediction*, 2019.

[53] A. Ploshkin *et al.*, "Yambda-5B — A Large-Scale Multi-modal Dataset for Ranking And Retrieval," *arXiv preprint arXiv:2505.22238*, 2025.

[54] A. Petrov, I. Safilo, D. Tikhonovich, and D. Ignatov, "MTS Kion Implicit Contextualised Sequential Dataset for Movie Recommendation," *arXiv preprint arXiv:2209.00325*, 2022.

[55] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *arXiv preprint arXiv:1711.05101*, 2017.

# A Configurations and cost

Table 23 lists what differs between the datasets; everything else is in `results/<session>/experiment_config.json`.

Table 23: What the sessions ran; for Yambda's final session, a first session and its continuation together. Fine-tuning: 512 lists per iteration, 4 gradient steps per batch, 2,000 iterations for the main run and 1,000 for every other run.

| | **KION** | **Yambda-50M** |
|---|---|---|
| Catalogue | titles with at least 10 training events | 50,000 most frequent tracks |
| Users | at least 5 training events | at least 50 training events |
| Window, dropout | 48 events, 0.2 | 128 events, 0.1 |
| Epochs: recommender / reward models | at most 30 / 15, early stopping | at most 14 / 8, early stopping |
| Session gap, return window | calendar day, 7 days | 30 minutes, 1 day |
| Reactions, adoption | not recorded | latest play within a day; 14 days |
| Items of the history in a list | blocked | allowed |
| *Final sessions* | | |
| Anchor weights | 0, 0.1, 0.2, 0.3, 0.5, 1 | 0, 0.03, 0.05, 0.1, 0.3 |
| KL budgets | 0.1, 0.5 | 0.2, 0.5 |
| Leave-one-out at weights | 0.1, 0.3 | 0.03, 0.1 |
| Credit ablations | weights 0.1, 0.3; both budgets | both budgets |
| Relative-share reward at weights | 0, 0.1, 0.3 | 0.03, 0.1 |
| Logged relevance | alone at 0; with the share at 0, 0.03, 0.1, 0.3 | alone at 0; with the share at 0, 0.1 |
| Sampled feedback, list reward at weights | 0.1, 0.3 | 0.1 |
| Retention rewards | session, return, share + return at 0.1, 0.3 | return at 0.1 |
| Personal rewards at weights | 0, 0.1, 0.3 | 0.1 |
| *Review sessions* | | |
| Anchor weights | 0, 0.1, 0.3, 1 | 0.05, 0.1, 0.3 |
| Long-term rewards at 0.1, 0.3 | return after the list, also with discount 0 | like − dislike, adoption, return after the list, also with discount 0 |
| Hidden responses: RL at weights | 0.1, 0.3 | 0.1, 0.3 |
| *Seed sessions (seeds 1 and 2)* | | |
| Anchor weights | 0.1, 0.3, 0.5, 1 | 0.03, 0.1, 0.3 |
| KL budget; leave-one-out; credit | 0.1 nats; at 0.3 and at 0.1 nats; at 0.1 nats | 0.5 nats; –; at 0.5 nats |
| Alternatives | liked-only, off-policy REINFORCE ($\kappa = 0.5$), re-ranking | the same |
| Rewards | logged relevance alone and with the share at 0.3; relative share at 0.3 | logged relevance alone |

Table 24: Wall-clock minutes by stage of the final sessions on one RTX PRO 6000 (96 GB). One fine-tuning run of 1,000 iterations takes 6.6 minutes on KION and 18.5 on Yambda, with its evaluation on test and validation requests. The review sessions took 99 and 419 minutes, the seed sessions 112 and about 270 minutes per seed.

| | **KION** | **Yambda, first session** | **Yambda, continuation** |
|---|---|---|---|
| Base recommender | 4 | 93 | 0 |
| Two reward models | 5 | 19 | 1 |
| Baselines | 8 | 112 | 2 |
| Main fine-tuning run | 17 | 39 | 0 |
| Anchor sweep | 39 | 56 | 39 |
| KL budgets | 13 | 18 | 19 |
| Advantage estimators | 27 | 45 | 17 |
| Sampled simulated feedback | 21 | 0 | 20 |
| Split of the gain, re-ranking | 6 | 2 | 2 |
| Alternatives to RL | 6 | 15 | 1 |
| Retention rewards | 38 | 0 | 18 |
| Logged relevance in the reward | 57 | 18 | 37 |
| Personal rewards | 44 | 1 | 41 |
| Relative-share reward | 22 | 38 | 2 |
| List reward | 19 | 0 | 21 |
| Credit ablations | 39 | 27 | 28 |

Table 25: Selection of the anchor weight on validation requests: the highest predicted share among the runs within 5% of the base recommender's NDCG@10.

| | **NDCG@10, validation** | **Pred. share, validation** | **Within tolerance** | **Selected** |
|---|---|---|---|---|
| *KION* | | | | |
| Base recommender | 0.1296 | 0.457 | yes | no |
| $\beta = 1$ | 0.1268 | 0.488 | yes | yes |
| $\beta = 0.5$ | 0.1223 | 0.519 | no | no |
| $\beta = 0.3$ | 0.1133 | 0.556 | no | no |
| $\beta = 0.2$ | 0.1072 | 0.578 | no | no |
| $\beta = 0.1$ | 0.0766 | 0.656 | no | no |
| $\beta = 0$ | 0.0004 | 0.689 | no | no |
| *Yambda* | | | | |
| Base recommender | 0.1314 | 0.599 | yes | no |
| $\beta = 0.3$ | 0.1314 | 0.608 | yes | no |
| $\beta = 0.1$ | 0.1313 | 0.616 | yes | no |
| $\beta = 0.05$ | 0.1273 | 0.628 | yes | yes |
| $\beta = 0.03$ | 0.0056 | 0.772 | no | no |
| $\beta = 0$ | 0.0015 | 0.809 | no | no |

# B Further results

Table 26: Fine-tuning with expected and with sampled simulated feedback, and the held-out predicted share of each policy under three assumptions about the response to earlier items of the list.

| | | | | **Held-out share when earlier items are** | | |
|---|---|---|---|---|---|---|
| | **KL** | **NDCG@10** | **Coverage** | **expected** | **sampled** | **finished** |
| *KION* | | | | | | |
| Expected feedback, $\beta = 0.3$ | 0.080 | 0.1043 | 0.436 | 0.553 | 0.617 | 0.695 |
| Sampled feedback, $\beta = 0.3$ | 0.087 | 0.1033 | 0.442 | 0.554 | 0.627 | 0.706 |
| Expected feedback, $\beta = 0.1$ | 0.590 | 0.0714 | 0.287 | 0.655 | 0.716 | 0.768 |
| Sampled feedback, $\beta = 0.1$ | 0.475 | 0.0730 | 0.287 | 0.645 | 0.717 | 0.773 |
| *Yambda* | | | | | | |
| Expected feedback, $\beta = 0.1$ | 0.102 | 0.1158 | 0.479 | 0.617 | 0.728 | 0.873 |
| Sampled feedback, $\beta = 0.1$ | 0.076 | 0.1178 | 0.489 | 0.614 | 0.728 | 0.871 |

Table 27: Fine-tuning against the reward model with expected responses for generated items and against the reward model trained with hidden responses, fed "unknown" for them (review sessions).

| | KL | NDCG@10 | on liked | Pred. share | Logged | Coverage |
|---|---|---|---|---|---|---|
| *KION* | | | | | | |
| Base recommender | 0.000 | 0.1173 | 0.1214 | 0.456 | 0.0173 | 0.495 |
| Expected feedback, $\beta = 0.3$ | 0.080 | 0.1043 | 0.1231 | 0.553 | 0.0175 | 0.433 |
| Unknown feedback, $\beta = 0.3$ | 0.078 | 0.1059 | 0.1248 | 0.551 | 0.0179 | 0.430 |
| Expected feedback, $\beta = 0.1$ | 0.627 | 0.0703 | 0.0898 | 0.659 | 0.0112 | 0.278 |
| Unknown feedback, $\beta = 0.1$ | 0.447 | 0.0812 | 0.1030 | 0.630 | 0.0137 | 0.301 |
| *Yambda* | | | | | | |
| Base recommender | 0.000 | 0.1172 | 0.1005 | 0.596 | 0.0669 | 0.519 |
| Expected feedback, $\beta = 0.3$ | 0.029 | 0.1180 | 0.1026 | 0.604 | 0.0688 | 0.510 |
| Unknown feedback, $\beta = 0.3$ | 0.029 | 0.1172 | 0.1017 | 0.603 | 0.0679 | 0.507 |
| Expected feedback, $\beta = 0.1$ | 0.100 | 0.1187 | 0.1033 | 0.611 | 0.0692 | 0.492 |
| Unknown feedback, $\beta = 0.1$ | 0.152 | 0.1135 | 0.0985 | 0.614 | 0.0664 | 0.467 |

Table 28: Hit rate by position of the list, for the base recommender's ranked and generated lists (review sessions).

| | Ranked list | | | Generated list | | |
|---|---|---|---|---|---|---|
| | All lists | After a miss | After a hit | All lists | After a miss | After a hit |
| *KION* | | | | | | |
| position 1 | 0.0885 | – | – | 0.0885 | – | – |
| position 2 | 0.0601 | 0.0569 | 0.093 | 0.0600 | 0.0550 | 0.112 |
| position 3 | 0.0469 | 0.0453 | 0.072 | 0.0443 | 0.0414 | 0.089 |
| position 5 | 0.0324 | 0.0312 | 0.064 | 0.0287 | 0.0273 | 0.063 |
| position 10 | 0.0213 | 0.0207 | 0.047 | 0.0160 | 0.0151 | 0.060 |
| *Yambda* | | | | | | |
| position 1 | 0.1667 | – | – | 0.1667 | – | – |
| position 2 | 0.1234 | 0.0790 | 0.345 | 0.1318 | 0.0640 | 0.471 |
| position 3 | 0.1006 | 0.0696 | 0.321 | 0.1116 | 0.0482 | 0.528 |
| position 5 | 0.0810 | 0.0628 | 0.253 | 0.0903 | 0.0399 | 0.539 |
| position 10 | 0.0599 | 0.0512 | 0.192 | 0.0566 | 0.0293 | 0.453 |

Table 29: The reward model on the users' real next five items: the logged share and the AUC with logged responses, the bias of the prediction under each assumption about earlier responses, and the AUC under expected and unknown responses (review sessions).

| | Real responses | | Bias of the prediction when earlier responses are | | | | Expected | Unknown |
|---|---|---|---|---|---|---|---|---|
| | Logged share | AUC | Expected | Sampled | Finished | Unknown | AUC | AUC |
| *KION* | | | | | | | | |
| step 1 | 0.487 | 0.757 | −0.010 | −0.010 | −0.010 | −0.007 | 0.757 | 0.756 |
| step 2 | 0.472 | 0.769 | −0.006 | 0.012 | 0.064 | 0.004 | 0.761 | 0.759 |
| step 3 | 0.466 | 0.777 | −0.013 | 0.019 | 0.098 | 0.004 | 0.761 | 0.762 |
| step 4 | 0.459 | 0.780 | −0.012 | 0.032 | 0.134 | 0.013 | 0.759 | 0.756 |
| step 5 | 0.460 | 0.782 | −0.019 | 0.035 | 0.153 | 0.010 | 0.760 | 0.759 |
| *Yambda* | | | | | | | | |
| step 1 | 0.617 | 0.640 | 0.063 | 0.063 | 0.063 | 0.070 | 0.640 | 0.640 |
| step 2 | 0.690 | 0.756 | −0.063 | 0.003 | 0.098 | 0.002 | 0.686 | 0.691 |
| step 3 | 0.690 | 0.788 | −0.072 | 0.016 | 0.137 | 0.003 | 0.688 | 0.694 |
| step 4 | 0.689 | 0.807 | −0.091 | 0.025 | 0.163 | 0.001 | 0.697 | 0.703 |
| step 5 | 0.686 | 0.819 | −0.093 | 0.032 | 0.183 | 0.009 | 0.687 | 0.702 |

Table 30: Yambda: the like and dislike heads of the held-out reward model on test events.

| | AUC | Base rate, % | Events |
|---|---|---|---|
| *Yambda, held-out reward model* | | | |
| Like | 0.736 | 0.75 | 914439 |
| Dislike | 0.729 | 0.12 | 914439 |

Table 31: Retention rewards. "Continues" and "returns" are the held-out model's predictions for the generated lists.

| | NDCG@10 | Pred. share | Continues | Returns | Coverage | Overlap |
|---|---|---|---|---|---|---|
| *KION* | | | | | | |
| Base recommender | 0.1173 | 0.455 | 0.345 | 0.718 | 0.495 | 0.209 |
| Session, $\beta = 0.3$ | 0.0951 | 0.438 | 0.404 | 0.754 | 0.567 | 0.078 |
| Session, $\beta = 0.1$ | 0.0443 | 0.297 | 0.545 | 0.764 | 0.526 | 0.106 |
| Return, $\beta = 0.3$ | 0.1077 | 0.459 | 0.368 | 0.739 | 0.521 | 0.141 |
| Return, $\beta = 0.1$ | 0.0903 | 0.466 | 0.392 | 0.765 | 0.544 | 0.093 |
| Share + return, $\beta = 0.3$ | 0.1091 | 0.502 | 0.355 | 0.733 | 0.491 | 0.183 |
| Share + return, $\beta = 0.1$ | 0.0943 | 0.546 | 0.372 | 0.756 | 0.477 | 0.167 |
| *Yambda* | | | | | | |
| Base recommender | 0.1172 | 0.599 | 0.947 | 0.913 | 0.519 | 0.001 |
| Return, $\beta = 0.1$ | 0.1158 | 0.600 | 0.947 | 0.913 | 0.516 | 0.002 |

Table 32: Logged relevance in the reward, alone and added to the share.

| | KL | NDCG@10 | on liked | Pred. share | Logged | Coverage | Overlap |
|---|---|---|---|---|---|---|---|
| *KION* | | | | | | | |
| Base recommender | 0.000 | 0.1173 | 0.1214 | 0.455 | 0.0173 | 0.495 | 0.209 |
| Logged relevance alone, $\beta = 0$ | 1.237 | 0.1180 | 0.1279 | 0.470 | 0.0172 | 0.097 | 0.518 |
| Share + 10 × logged relevance, $\beta = 0.3$ | 0.080 | 0.1194 | 0.1303 | 0.495 | 0.0185 | 0.426 | 0.282 |
| Share + 10 × logged relevance, $\beta = 0.1$ | 0.276 | 0.1158 | 0.1323 | 0.536 | 0.0188 | 0.345 | 0.334 |
| Share + 10 × logged relevance, $\beta = 0.03$ | 0.728 | 0.1078 | 0.1305 | 0.581 | 0.0181 | 0.222 | 0.428 |
| Share + 10 × logged relevance, $\beta = 0$ | 1.914 | 0.0952 | 0.1203 | 0.649 | 0.0152 | 0.106 | 0.624 |
| Share + 3 × logged relevance, $\beta = 0.3$ | 0.080 | 0.1130 | 0.1287 | 0.532 | 0.0184 | 0.421 | 0.269 |
| Share + 3 × logged relevance, $\beta = 0.1$ | 0.311 | 0.1004 | 0.1241 | 0.595 | 0.0170 | 0.330 | 0.350 |
| Share + 3 × logged relevance, $\beta = 0.03$ | 1.130 | 0.0854 | 0.1097 | 0.674 | 0.0134 | 0.162 | 0.518 |
| Share + 3 × logged relevance, $\beta = 0$ | 2.059 | 0.0783 | 0.1020 | 0.680 | 0.0128 | 0.084 | 0.651 |
| *Yambda* | | | | | | | |
| Base recommender | 0.000 | 0.1172 | 0.1005 | 0.599 | 0.0669 | 0.519 | 0.001 |
| Logged relevance alone, $\beta = 0$ | 1.012 | 0.1437 | 0.1259 | 0.604 | 0.0836 | 0.486 | 0.004 |
| Share + 10 × logged relevance, $\beta = 0.1$ | 0.237 | 0.1309 | 0.1139 | 0.606 | 0.0758 | 0.516 | 0.002 |
| Share + 10 × logged relevance, $\beta = 0$ | 0.978 | 0.1433 | 0.1245 | 0.608 | 0.0830 | 0.501 | 0.004 |

Table 33: Yambda: relevance on tracks from the visible history and on new tracks.

| | NDCG@10 | on repeats | on new | From history | Logged | Coverage |
|---|---|---|---|---|---|---|
| Replay of the history | 0.1873 | 0.3722 | 0.0000 | 100% | 0.1103 | 0.519 |
| Base recommender | 0.1172 | 0.1865 | 0.0261 | 51% | 0.0669 | 0.519 |
| RL on the share, $\beta = 0.3$ (main run) | 0.1183 | 0.1868 | 0.0271 | 51% | 0.0686 | 0.506 |
| Liked-only fine-tuning | 0.1198 | 0.1903 | 0.0269 | 51% | 0.0692 | 0.515 |
| Off-policy REINFORCE, $\varkappa = 0.5$ | 0.1245 | 0.1999 | 0.0261 | 49% | 0.0735 | 0.475 |
| Re-ranking, w = 0.3 | 0.1155 | 0.1786 | 0.0321 | 43% | 0.0659 | 0.456 |
| RL on logged relevance alone, $\beta = 0$ | 0.1437 | 0.2487 | 0.0201 | 62% | 0.0836 | 0.486 |
| RL on share + 10 × logged relevance, $\beta = 0.1$ | 0.1309 | 0.2142 | 0.0250 | 55% | 0.0758 | 0.516 |

Table 34: Rewards with a baseline. "Personal part" is the predicted share for the user minus that for another user.

| | KL | NDCG@10 | Pred. share | Personal part | Coverage | Overlap |
|---|---|---|---|---|---|---|
| *KION* | | | | | | |
| Base recommender | 0.000 | 0.1173 | 0.455 | 0.014 | 0.495 | 0.209 |
| Personal share, $\beta = 0.3$ | 0.022 | 0.1161 | 0.455 | 0.019 | 0.487 | 0.205 |
| Personal share, $\beta = 0.1$ | 0.097 | 0.1135 | 0.448 | 0.024 | 0.453 | 0.213 |
| Personal share, $\beta = 0$ | 1.168 | 0.0686 | 0.313 | 0.029 | 0.336 | 0.332 |
| Share above the item mean, $\beta = 0.3$ | 0.187 | 0.1066 | 0.407 | 0.021 | 0.406 | 0.225 |
| Share above the item mean, $\beta = 0.1$ | 0.980 | 0.0613 | 0.368 | 0.026 | 0.239 | 0.262 |
| Share above the item mean, $\beta = 0$ | 4.183 | 0.0327 | 0.424 | 0.002 | 0.038 | 0.797 |
| *Yambda* | | | | | | |
| Base recommender | 0.000 | 0.1172 | 0.599 | 0.101 | 0.519 | 0.001 |
| Personal share, $\beta = 0.1$ | 0.054 | 0.1166 | 0.603 | 0.106 | 0.516 | 0.002 |
| Share above the item mean, $\beta = 0.1$ | 0.315 | 0.1056 | 0.590 | 0.109 | 0.437 | 0.002 |

Table 35: A list-dependent reward: predicted share plus the mean distance between the items of the list in the base recommender's embedding space. "List reward" is the reward per item as the held-out model sees it. Greedy re-ranking is maximal marginal relevance over 100 candidates.

| | **NDCG@10** | **Pred. share** | **Distance** | **List reward** | **Coverage** |
|---|---|---|---|---|---|
| *KION* | | | | | |
| Base recommender | 0.1173 | 0.455 | 0.617 | 0.764 | 0.495 |
| RL on the list reward, $\beta = 0.3$ | 0.1111 | 0.520 | 0.655 | 0.848 | 0.455 |
| RL on the list reward, $\beta = 0.1$ | 0.0991 | 0.583 | 0.699 | 0.932 | 0.394 |
| Greedy re-ranking, w = 1 | 0.1201 | 0.499 | 0.692 | 0.845 | 0.580 |
| Greedy re-ranking, w = 0.3 | 0.1104 | 0.550 | 0.749 | 0.925 | 0.571 |
| Greedy re-ranking, w = 0.1 | 0.0899 | 0.601 | 0.829 | 1.015 | 0.518 |
| Best of 8 sampled lists | 0.1060 | 0.495 | 0.665 | 0.828 | 0.511 |
| RL on the share alone, $\beta = 0.3$ | 0.1043 | 0.553 | 0.636 | 0.871 | 0.436 |
| RL on the share alone, $\beta = 0.1$ | 0.0714 | 0.655 | 0.622 | 0.966 | 0.287 |
| *Yambda* | | | | | |
| Base recommender | 0.1172 | 0.599 | 0.715 | 0.957 | 0.519 |
| RL on the list reward, $\beta = 0.1$ | 0.1182 | 0.611 | 0.732 | 0.976 | 0.509 |
| Greedy re-ranking, w = 1 | 0.1145 | 0.613 | 0.744 | 0.984 | 0.465 |
| Greedy re-ranking, w = 0.3 | 0.1155 | 0.617 | 0.769 | 1.002 | 0.462 |
| Greedy re-ranking, w = 0.1 | 0.1105 | 0.624 | 0.821 | 1.034 | 0.447 |
| Best of 8 sampled lists | 0.1147 | 0.611 | 0.761 | 0.991 | 0.516 |
| RL on the share alone, $\beta = 0.3$ | 0.1176 | 0.608 | 0.706 | 0.961 | 0.505 |
| RL on the share alone, $\beta = 0.1$ | 0.1158 | 0.617 | 0.694 | 0.964 | 0.479 |

# C Further figures

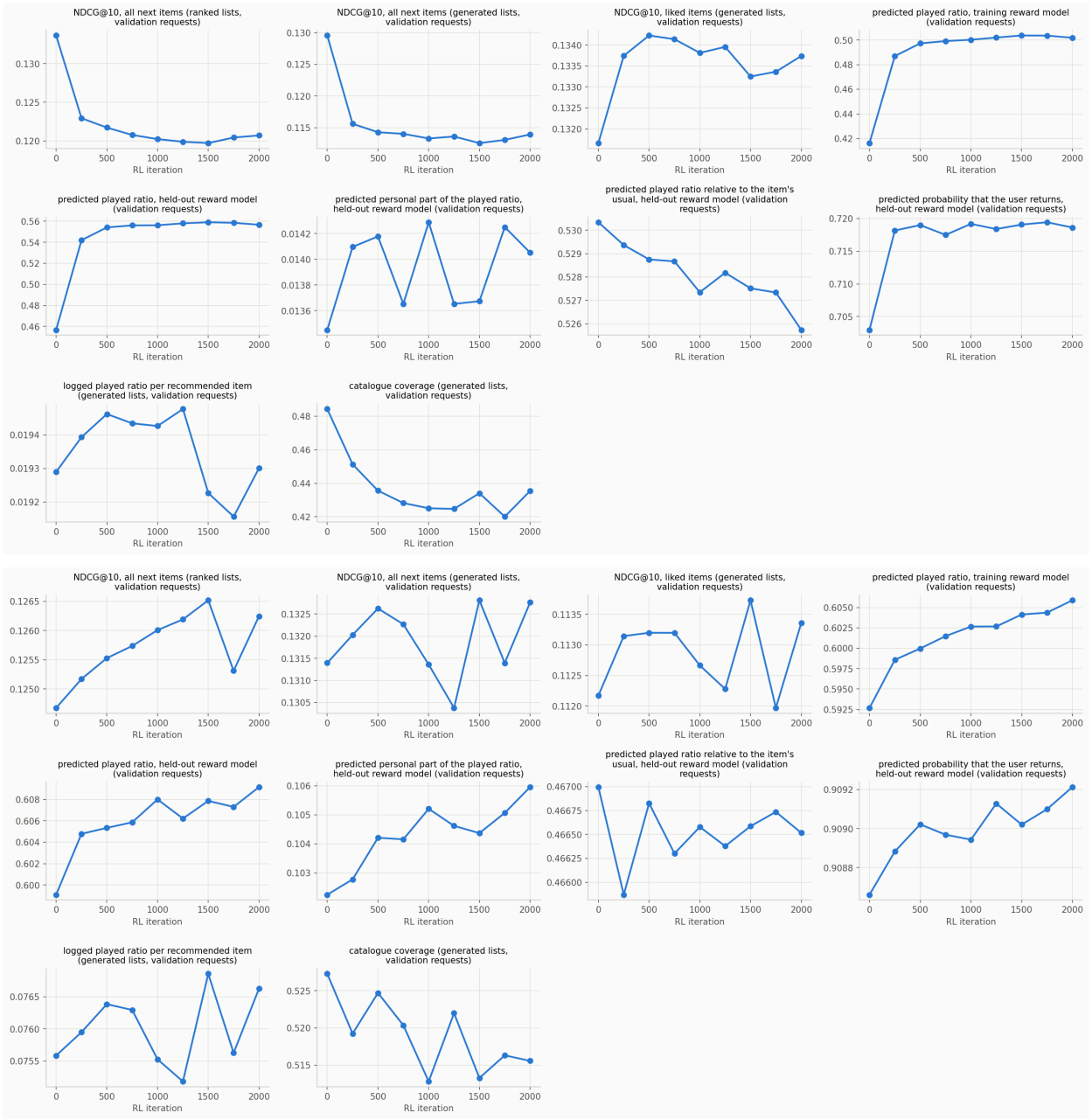



Figure 6: The main run on validation requests during fine-tuning: KION (top) and Yambda (bottom).

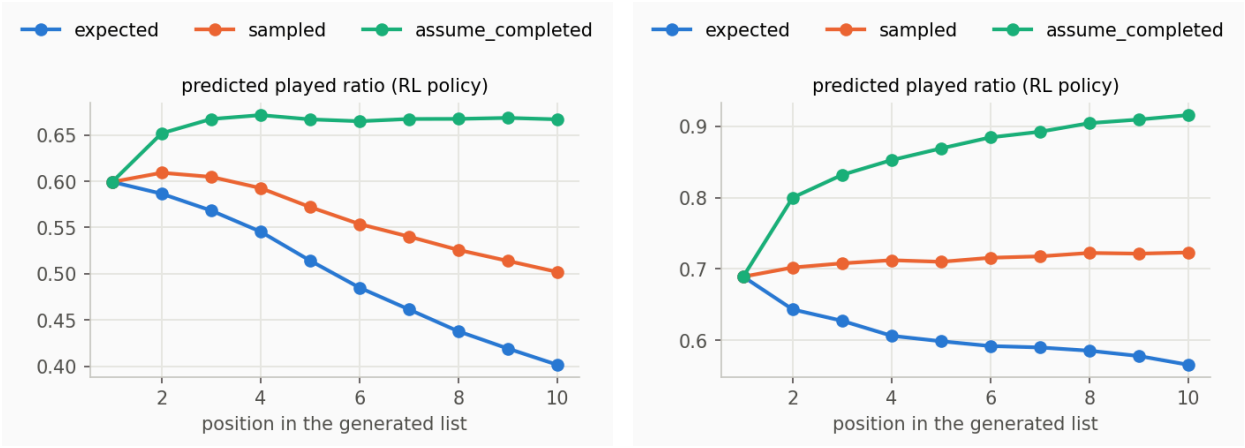



Figure 7: Predicted share by position of the list for the base recommender under three assumptions about the response to the items before it: KION (left) and Yambda (right).

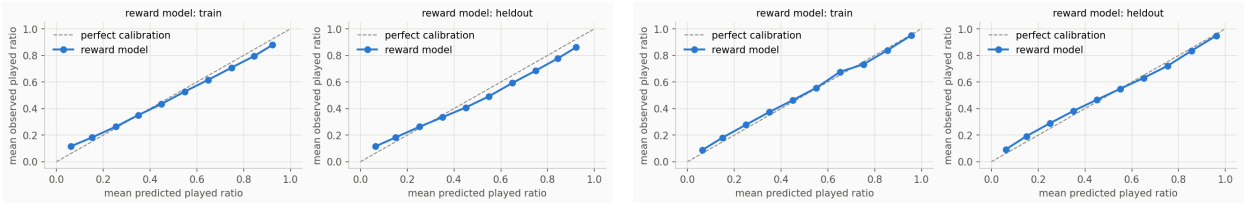



Figure 8: Calibration of the two reward models on test events: KION (left) and Yambda (right).

# D Earlier iterations of the study

The pipeline was completed seven times before the final sessions (version 2 of this draft describes those runs in full). They differ from the present sessions in the reward models, which had fewer heads, and in the instruments available, and they agree with them where they overlap. The main run on KION was made three times with the same base recommender and other reward models: NDCG@10 changed by $-11.4\%$, $-11.4\%$ and $-11.1\%$ and the held-out predicted share by $+22.8\%$, $+23.1\%$ and $+23.1\%$, within the spread of the three seeds here ($-11.5\% \pm 1.3$ and $+23.9\% \pm 2.1$). On Yambda two earlier runs gave $+1.1\%$ and $+1.8\%$ of predicted share, against $+1.67\% \pm 0.01$ here. The first two runs used REINFORCE with a leave-one-out baseline on Yambda, where the policy held at $\beta = 0.3$ and collapsed at $\beta = 0.1$, while the value-model runs held at 0.1 and collapsed at 0.03; Table 15 repeats the comparison with the learning rate and the number of lists per history held equal and confirms that the estimator alone moves the transition. Two findings of the earlier runs are not repeated here: a choice head trained to tell logged items from popularity-distributed replacements did not work as a relevance signal inside the reward, and a reward weighted by item duration moved the policy along the duration axis in the direction the normalisation favoured.